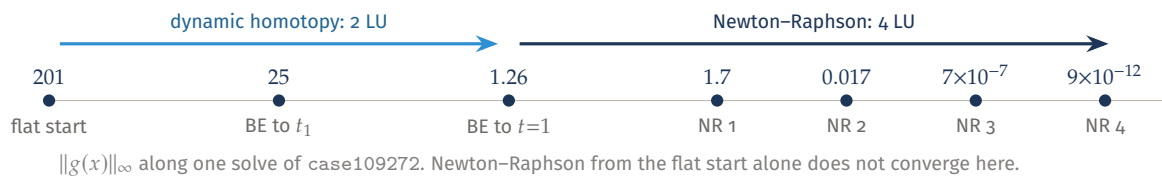




DynHomotopy

Getting Newton–Raphson to converge from a flat start
on ill-conditioned power grids

*A reproduction and audit of a dynamic-homotopy method, eight
modifications and four numerical studies on 34 power systems of up to 109,272 buses*



SUBMITTED BY

2105039 Mohiuddin Sizan
2105040 Sadman bin Tareq
2105057 Mosharaf Hossain Apurbo
2105058 Elias Hasbi
2105060 Jonayed Mohiuddin Shanto

SUBMITTED TO

Anup Halder Joy
Department of CSE, BUET

BASE PAPER

A. Lima-Silva and F.D. Freitas, *Energies*
17(18):4642, 2024.

Abstract

Newton–Raphson (NR) is the standard way to solve the power flow equations of an electrical grid, but on large, badly conditioned networks it diverges when started from the usual *flat start* (every voltage at 1 pu, every angle at zero). Lima-Silva and Freitas (2024) proposed a remedy: embed the equations in a homotopy $G(x, t) = t g(x) + (1 - t)K(x - x^{(0)})$, turn it into an ordinary differential equation in t , integrate that ODE with only two or three steps of backward Euler, and hand the rough result to NR or the fast decoupled method. This report describes **DynHomotopy**, our Python implementation of that method from first principles, and what we learned by pushing on it.

We rebuilt the complete pipeline (MATPOWER case reader, admittance matrix, polar power-flow equations with an analytic Jacobian, NR, fast decoupled XB, the paper’s three integrators and our reconstruction of its static-homotopy baseline) and reproduced every table and figure of the paper. Of the 255 mismatch norms printed in the paper’s Tables 2–5, 246 agree with ours to within a factor of two, and 54 of the 63 iteration counts in its Table 7 agree exactly; the remaining differences trace back to eight inconsistencies we found in the paper itself, which we document. We show algebraically that the paper’s recommended two-point backward-Euler path is a shifted (Levenberg–Marquardt-like) Newton step followed by an almost plain Newton step, which explains both why it works and why explicit integrators cannot.

We then added eight switchable modifications and tested 17 configurations on a test bed widened from the paper’s 9 systems to 34, under five parameter settings, for 2,890 runs, with cost counted in LU factorizations rather than seconds. A run counts as solved only if it reaches the *reference* operating point: 14 of the 141 runs on which the paper’s method converges land on a different root of the power flow equations, so it truly solves 127 of 170. A Newton corrector after every path step raises this to 152 with no losses and almost removes the wrong-root problem (2 runs instead of 14), for 5% more factorizations; combined with Iwamoto’s optimal multiplier it keeps that robustness at 7% *fewer* factorizations than the paper’s method, which makes multiplier plus corrector our recommended configuration. Richardson error control of the path steps is the only change that solves some systems nothing else solves, at 18–30% more factorizations. Four other ideas—a chord refinement of backward Euler that the paper itself suggests, fixed-step RK4, a Jacobian-scaled homotopy and the standard Newton homotopy—make things worse, and we explain why from the equations.

Finally we carried out the numerical studies of our proposal: the spectrum of the flat-start Jacobian by the power method and shifted inverse iteration, maps of where the method works in the $(K, \Delta t_0)$ plane, golden-section tuning of the shift $\delta = K/\Delta t_0$, and Gauss elimination and LU factorization written from scratch. They show that δ matters only on the ill-conditioned grids, that no simple spectral rule predicts it, and why a sparse direct solver is indispensable at this scale.

Contents

Abstract	i
1 Introduction	1
1.1 The problem: finding where a grid settles	1
1.2 Problem statement	1
1.3 What we built	1
1.4 Scope	2
1.5 How this report is organised	3
2 Background and the Base Paper	3
2.1 The power balance equations	3
2.2 Newton–Raphson and the fast decoupled method	3
2.3 Homotopy methods	3
2.4 The base paper	4
2.5 Related ideas we build on	4
3 Test Systems	4
3.1 Where the systems come from	4
3.2 Structure of the Jacobian	5
3.3 Four details that change numbers	5
4 Methods Implemented from the Base Paper	7
4.1 Software architecture	7
4.2 The homotopy and its three integrators	7
4.3 What the first step really does	8
4.4 The refining solvers	9
4.5 Verification before measurement	10
5 Reproducing the Base Paper	11
5.1 The two-variable tutorial	11
5.2 The first time step	13
5.3 Tables 2–4: one large step after t_2	13
5.4 Table 5: growing steps	14
5.5 How close is the reproduction, overall?	14
5.6 Section 4.4: larger first steps	14
5.7 Figure 3 and Table 6: five-point paths	16
5.8 Figures 4 and 5: the states themselves	17
5.9 Table 7: iterations	17
5.10 Table 8: run time	17
6 Auditing the Base Paper	19

7	Our Modifications	20
7.1	From proposal to implementation	20
7.2	Design rule: switches on top of an untouched reproduction	20
7.3	Modification 1: the optimal multiplier in NR	22
7.4	Modification 2: a Newton corrector after every step	22
7.5	Modification 3: Richardson error control of the path steps	22
7.6	Modification 4: choosing the time points by the mismatch	23
7.7	Modifications 5–8: four ideas that did not work	23
7.8	Evaluation protocol	23
8	Results of the Modifications	24
8.1	First, which root? The paper’s method on the wider test bed	24
8.2	Headline numbers	24
8.3	Robustness against cost	26
8.4	Richardson step control	26
8.5	System by system	27
8.6	Rescues, root fixes and speed-ups	27
9	The Proposal’s Numerical Studies	30
9.1	The spectrum of the flat-start Jacobian	30
9.2	Where the method works in the $(K, \Delta t_0)$ plane	31
9.3	Tuning δ by golden-section search	32
9.4	Linear solvers written from scratch	33
10	Failure Analysis	34
10.1	Why explicit schemes fail on the first step	34
10.2	Why explicit schemes still fail with large later steps	34
10.3	Why RK4 fails, and why higher order does not pay under error control	34
10.4	Why refining backward Euler makes it worse	35
10.5	Why the Newton homotopy fails	35
10.6	Why the scaled homotopy changes little	35
10.7	Why part of the corrector’s gain is accounting	35
10.8	Where adaptive steps lose	36
10.9	Why Richardson control still finds wrong roots	36
10.10	Why no simple rule predicts δ	36
10.11	The seven runs nothing solves	36
10.12	Failures of our own measurement	36
11	Discussion	37
11.1	Strengths	37
11.2	Limitations	37
11.3	Future work	37
12	Conclusion	38

A	Reproducing This Report	40
B	Full Tables	40

List of Figures

1	The method of the base paper at a glance	2
2	Size and flat-start mismatch of the test systems	5
3	Jacobian sparsity at the flat start	7
4	Architecture of the implementation	8
5	What each integrator sees on the first step	9
6	Geometry of the dynamic homotopy	10
7	The tutorial problem in the plane	11
8	Tutorial: our versions of the paper’s Figures 1 and 2	12
9	First step: explicit schemes explode	13
10	Reproduction fidelity across Tables 2–5	15
11	Section 4.4 re-run: which paths give a usable start	16
12	Figure 3 re-run: five-point paths	16
13	Figure 4 re-run: voltages of case109272	17
14	Figure 5 re-run: the cost of smaller steps	17
15	Figure 5 re-run: bus 6 of case109272 for five time steps	18
16	Table 8 re-run: run time relative to NR-MAT	19
17	From proposal to implementation	21
18	Where each of the eight switches acts	21
19	Evaluation protocol for the modifications	24
20	Runs solved per configuration, by setting	25
21	Robustness against cost	26
22	Richardson-controlled paths on case6024	27
23	LU factorizations per system and configuration, setting S1	28
24	A rescue, a root fix and a speed-up	29
25	The adaptive path on case6748	30
26	Spectrum of the flat-start Jacobian	31
27	Feasible region in the $(K, \Delta t_0)$ plane	32
28	Tuned shift against the spectrum	33

List of Tables

1	The 34 test systems	6
2	Tutorial: NR iterations after each start	13
3	Table 4 re-run: BE then NR	14
4	Table 5 re-run: growing time steps	15
5	Table 7 re-run: iterations of every method	18
6	Eight places where the paper is inconsistent or under-specified	20
7	Table 7 on the stored 10 MVA base	20
8	Runs solved by each configuration	25

9 Choosing δ : rules fitted on the paper’s systems 33

10 The paper’s method with our own LU 34

11 Map from each result to its script 40

12 Table 2 re-run: FE then NR 41

13 Table 3 re-run: RK2 then NR 41

14 Table 6 re-run: five-point paths, run time 42

1 Introduction

1.1 The problem: finding where a grid settles

Whenever an operator asks how the voltages across a network will settle for a given pattern of generation and demand—before switching a line out, after a generator trips, while planning next year’s load—a *power flow* problem is solved. For every bus k the power injected by generators minus the power drawn by loads has to equal the power flowing out through the lines, which gives two nonlinear equations per bus in the unknown voltage magnitudes V and angles θ . Written compactly,

$$g(x) = 0, \quad x = [\theta_{PV}, \theta_{PQ}, V_{PQ}], \quad g : \mathbb{R}^n \rightarrow \mathbb{R}^n,$$

where n is 22 for the smallest system in this report and 185,807 for the largest.

The workhorse solver is Newton–Raphson (NR): linearise g at the current point, solve the resulting sparse linear system with one LU factorization, step, repeat. Near the solution it converges quadratically, typically in three to six iterations. Far from the solution it has no guarantee at all. And in practice it is usually started far away, from the *flat start*: every magnitude at 1 pu and every angle at zero, because nothing better is known after a large disturbance or for a network that has never been solved. On the five large, poorly conditioned grids of the base paper, NR from the flat start does not converge on any of them (Table 1), even though a solution exists and NR finds it easily from the stored operating point.

1.2 Problem statement

Lima-Silva and Freitas [1] proposed to replace the flat start with a better one, computed cheaply by following a homotopy from an easy problem to the real one with a very coarse numerical integrator. Their paper reports that backward Euler with only two path points is enough for NR to converge in four iterations on a 109,272-bus grid. This project set out to answer four questions:

- Q1. Does it reproduce?** If we build the method ourselves, from the equations in the paper and nothing else, do we get the paper’s numbers?
- Q2. Why does it work?** The paper motivates the choice of backward Euler over forward Euler and RK2 mostly by experiment. Is there a reason visible in the algebra?
- Q3. Is the paper internally consistent?** Where our numbers disagree, is the fault ours, or the paper’s?
- Q4. Can it be improved, and at what cost?** Which changes rescue systems the method fails on, which only make it faster, and which make it worse—measured on more systems than the paper uses, in a cost unit that does not depend on the computer, and counting a run as solved only if it reaches the right solution?
- Q5. What do the numerics say about its parameters?** What does the spectrum of the flat-start Jacobian look like, where in the $(K, \Delta t_0)$ plane does the method work, and can the shift δ be tuned or predicted?

1.3 What we built

- **dynhomotopy**, a Python package that implements the full method from first principles: MATPOWER case files, the admittance matrix with taps and phase shifters, the polar power-flow equations with an analytic Jacobian checked against finite differences, NR, the XB fast decoupled method (FDXB), our reconstruction of the paper’s static-homotopy

baseline GSH-NR, the fixed-point homotopy with forward Euler, RK2 and backward Euler, and path schedules (Section 4).

- **A complete reproduction:** one script per table or figure of the paper, every output cell printed as *ours* / *paper* (Section 5).
- **An audit** of eight places where the paper is inconsistent or under-specified (Section 6).
- **improvements**, a separate package of eight switchable modifications that never edits the reproduced code, and an evaluation of 17 configurations on 34 systems $\times$ 5 settings with LU factorizations as the cost unit and a check that each run reaches the reference operating point (Sections 7 and 8).
- **extensions**, the numerical studies promised in our proposal: power method and shifted inverse iteration for the Jacobian spectrum, $(K, \Delta t_0)$ feasibility maps, golden-section tuning of δ , and Gauss elimination and LU with partial pivoting written from scratch (Section 9).
- **A failure analysis** explaining from the equations why each unsuccessful modification fails (Section 10).
- **Two command-line tools**, `python -m improvements` and `python -m extensions`, that run any modification or study on any of the 34 systems.

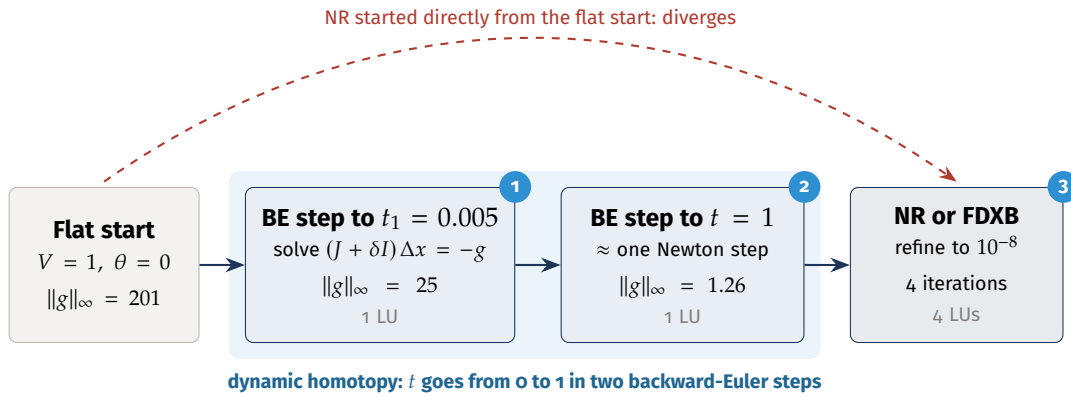


Figure 1. The method of the base paper at a glance, with the real mismatch norms of one solve of case109272 (Figure 13). Stage 1 regularises the flat-start Jacobian by $\delta = K/\Delta t_0$; stage 2 is almost a plain Newton step; stage 3 is ordinary NR from a much better starting point. Section 4.3 derives both interpretations.

1.4 Scope

We treat steady-state AC power flow on transmission models in MATPOWER format, with the slack bus fixed and generator reactive limits not enforced (MATPOWER’s default, and the paper’s setting). Distribution networks, optimal power flow and contingency analysis are outside the scope. All reported experiments solve their linear systems with SuperLU through SciPy [19, 20]: the systems reach 185,807 unknowns and 1.4 million nonzeros. We also wrote Gauss elimination and LU with partial pivoting from scratch and ran the paper’s method with them on the systems up to 2,000 buses (Section 9.4); they give the same iterates, and they show in numbers why a dense solver cannot be used on the large grids. Run times are reported, but the primary cost measure is the number of LU factorizations, because our Python timings are not comparable with the paper’s MATLAB timings.

1.5 How this report is organised

Section 2 reviews the power flow equations, NR, FDXB and homotopy methods, and the base paper's claims. Section 3 describes the 34 test systems. Section 4 presents the implementation and derives what the first and last integration steps actually do. Section 5 reproduces the paper result by result, and Section 6 lists where the paper and our results disagree and why. Section 7 introduces our eight modifications and the evaluation protocol, Section 8 reports their results, Section 9 presents the numerical studies of the proposal, and Section 10 is a dedicated failure analysis. Sections 11 and 12 close with discussion and conclusions; the appendices give reproduction commands and the full tables.

2 Background and the Base Paper

2.1 The power balance equations

Let $Y = G + jB$ be the bus admittance matrix of the network and $V_k \angle \theta_k$ the voltage at bus k . The active and reactive mismatch at bus k are

$$\Delta P_k = V_k \sum_m V_m (G_{km} \cos \theta_{km} + B_{km} \sin \theta_{km}) - P_k^{\text{sp}}, \quad (1)$$

$$\Delta Q_k = V_k \sum_m V_m (G_{km} \sin \theta_{km} - B_{km} \cos \theta_{km}) - Q_k^{\text{sp}}, \quad (2)$$

with $\theta_{km} = \theta_k - \theta_m$. Buses come in three types. At the single *slack* bus V and θ are fixed. At *PV* buses (generators) P and V are specified and θ is unknown. At *PQ* buses (loads) both V and θ are unknown. Collecting the unknowns gives the state $x = [\theta_{\text{PV}}, \theta_{\text{PQ}}, V_{\text{PQ}}] \in \mathbb{R}^n$ with $n = N_{\text{PV}} + 2N_{\text{PQ}}$, and the equations $g(x) = [\Delta P_{\text{PV}}, \Delta P_{\text{PQ}}, \Delta Q_{\text{PQ}}] = 0$. In complex form the whole mismatch vector is $\Delta S = V \circ (\overline{YV}) - S^{\text{sp}}$, which is how our implementation evaluates it (the same ordering and sign as MATPOWER [10]).

2.2 Newton–Raphson and the fast decoupled method

NR updates $x^{(i+1)} = x^{(i)} - J(x^{(i)})^{-1} g(x^{(i)})$, where $J = \partial g / \partial x$ is sparse with the sparsity pattern of the network [7]. Each iteration costs one LU factorization of J . The fast decoupled method [8] approximates J by two constant matrices,

$$\Delta P/V = B' \Delta \theta, \quad \Delta Q/V = B'' \Delta V, \quad (3)$$

factorised once and reused. In the XB variant [9] used by the paper, resistances are ignored only when building B' . FDXB iterations are cheap but many. The paper's Table 7 counts every P – θ update and every Q – V update as one iteration; MATPOWER's `fdpf` instead returns one count per P/Q cycle and only prints the P and Q counts separately.

2.3 Homotopy methods

A homotopy replaces the hard problem $g(x) = 0$ by a family $G(x, t) = 0$, $t \in [0, 1]$, whose member at $t = 0$ is trivial and whose member at $t = 1$ is the real problem [14, 15]. A *static* homotopy picks points $0 < t_1 < \dots < t_N = 1$ and solves $G(x, t_k) = 0$ accurately at each one with NR, using the previous solution as the start. The GSH-NR method of [3], the baseline in the base paper, is of this kind: it adds fictitious shunts that make the flat start an exact solution at $t = 0$ and removes them gradually. Its drawback is cost: every path point is a full NR solve.

A *dynamic* homotopy [2] instead differentiates $G(x(t), t) = 0$ along the path,

$$G_x \frac{dx}{dt} + G_t = 0 \implies \frac{dx}{dt} = -G_x(x, t)^{-1} G_t(x, t), \quad x(0) = x^{(0)}, \quad (4)$$

and *integrates* this ODE numerically, accepting that the result at $t = 1$ is only approximate. Continuous Newton methods [6] are a close relative: they treat NR itself as the forward Euler discretisation of an ODE.

2.4 The base paper

Lima-Silva and Freitas [1] use the fixed-point homotopy

$$G(x, t) = t g(x) + (1 - t) K (x - x^{(0)}), \quad (5)$$

whose derivatives are $G_x = t J(x) + (1 - t) KI$ and $G_t = g(x) - K(x - x^{(0)})$. The factor K , which earlier work had set to one, is their main design lever. They integrate (4) with forward Euler (FE), RK2 and a backward Euler (BE) scheme simplified to a single fixed-point iteration, always take the first step to $t_1 = \Delta t_0$ with BE, and refine the result at $t = 1$ with NR or FDXB. On five ill-conditioned grids built by replicating PEGASE networks [13] (18,482 to 109,272 buses) and four loading-limit cases [11, 12], they report that:

- FE and RK2 with large steps blow up on the big systems, while BE succeeds everywhere (their Tables 2–4);
- BE with just two path points, $\{0, 0.005, 1\}$, lets NR converge in four iterations where NR from the flat start fails (Table 7);
- BE followed by FDXB is faster than NR from the stored operating point, at 57–85% of its CPU time (Table 8).

The paper’s own future work mentions other homotopy functions and other integrators, which is where two of our modifications come from.

2.5 Related ideas we build on

Iwamoto and Tamura [5] introduced the *optimal multiplier*: scale each NR step by the factor μ that minimises the mismatch along it, which for power flow in rectangular coordinates is an exact quadratic. Predictor–corrector continuation [15] alternates a cheap predictor step along the path with a Newton corrector back onto it. Adding a multiple of the identity to an ill-conditioned Jacobian is the idea behind Levenberg–Marquardt damping [17, 18], and behind the conditioning step of [4], which the base paper cites for its δI term. The implicit Euler method’s stability on stiff problems is textbook material [16]; Section 4.3 shows it is exactly what matters here.

3 Test Systems

3.1 Where the systems come from

All systems are MATPOWER cases from two public Zenodo records by Tostado-Véliz, Kamel and Jurado, the same files the paper uses: record 3514739 holds the *ill-conditioned* systems [11], built by gluing copies of European PEGASE and Polish networks together, and record 3491654 holds the *loading-limit* systems [12], in which every load of a standard case was scaled up until NR from a flat start was about to stop converging. Our code downloads each file on first use.

The paper uses nine of these systems. For the modification study we added every other system in the two records that loads, giving 34: the paper’s 5 ill-conditioned and 4 limit cases, 7 more ill-conditioned systems and 18 more limit cases (Table 1 and Figure 2). The wider test bed matters: 25 of the 34 systems were never seen while the paper’s parameters were being chosen, so they test whether those parameters generalise.

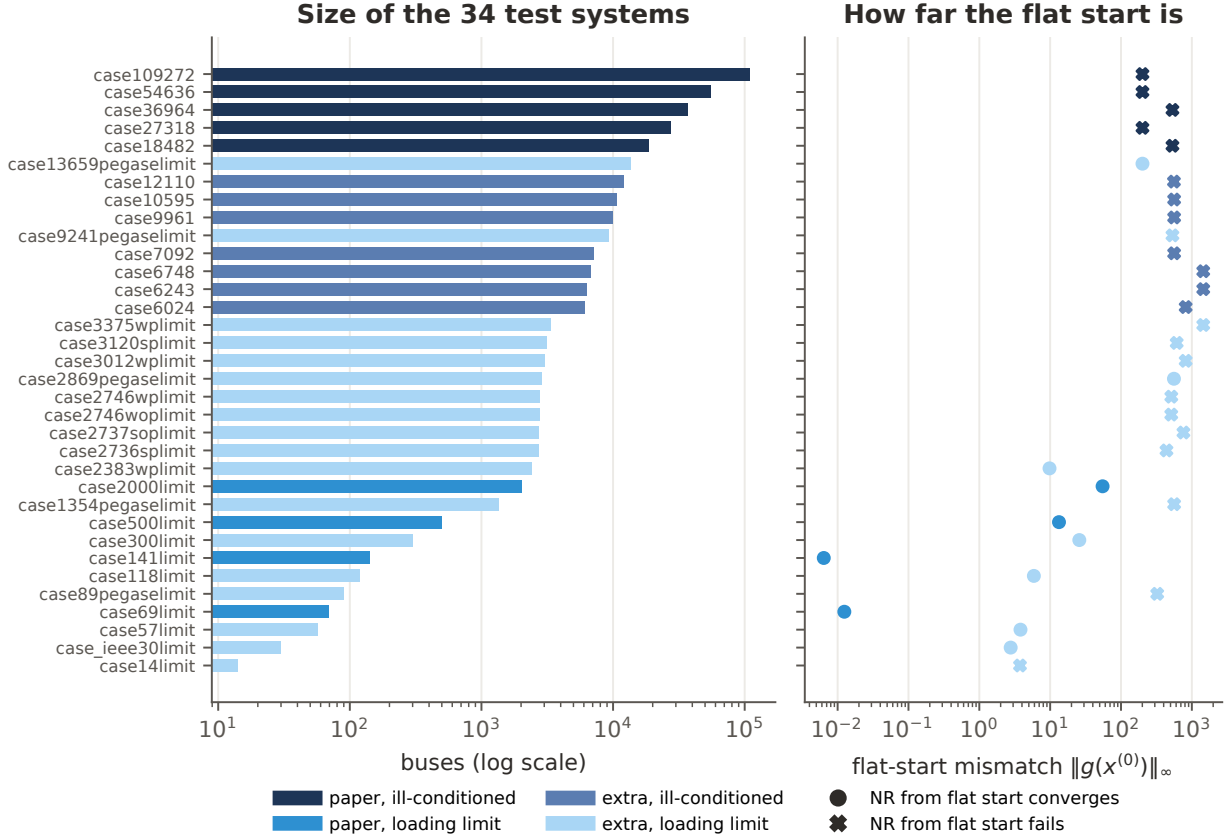


Figure 2. Size and flat-start mismatch of all 34 test systems, sorted by size. Size alone does not decide difficulty: NR from the flat start fails on the 14-bus limit case and succeeds on the 13,659-bus one. What the failing systems share is a large initial mismatch—several hundred per unit on the ill-conditioned replicas.

3.2 Structure of the Jacobian

Figure 3 shows the Jacobian at the flat start for three systems. The replicated construction is directly visible: case18482 is two copies of case9241pegase, and case109272 is eight copies of case13659pegase, each copy one diagonal block with only a few coupling entries between them. Two consequences run through the rest of the report. First, the flat-start mismatch of a replica equals that of its base network (533 for every 9241-based system, 201 for every 13659-based one). Second, case54636 and case109272, which are four and eight copies of the same network, behave *identically* in every experiment—they give the same norms to every printed digit. That identity turns out to be a useful check on the paper (Section 6).

3.3 Four details that change numbers

Base MVA of two limit cases. case69limit and case141limit are stored with baseMVA = 10. The paper’s Tables 2–5 for these cases come out only on a 100 MVA base, so every experiment uses 100 for them—but its NR-MAT and NR-flat counts in Table 7 match the stored 10. We return to this in Section 6.

Table 1. The 34 test systems. n is the number of unknowns, nnz the nonzeros of the Jacobian at the flat start, and the last column the NR iterations needed from the flat start (limit 10). The replicated construction of the ill-conditioned systems is from the dataset’s own description.

Case	Built from	Buses	PV	n	nnz(J)	$\ g(x^{(0)})\ _{\infty}$	NR flat
<i>Paper, ill-conditioned (record 3514739)</i>							
case18482	9241pegase x2	18,482	2,889	34,073	258,947	533	fail
case27318	13659pegase x2	27,318	8,183	46,451	349,411	201	fail
case36964	9241pegase x4	36,964	5,779	68,147	517,923	533	fail
case54636	13659pegase x4	54,636	16,367	92,903	698,829	201	fail
case109272	13659pegase x8	109,272	32,735	185,807	1,397,667	201	fail
<i>Paper, loading limit (record 3491654)</i>							
case69limit		69	0	136	808	0.0124	2
case141limit		141	0	280	1,672	6.4e-03	2
case500limit		500	55	943	6,275	13.3	4
case2000limit		2,000	391	3,607	26,345	55	5
<i>Added by us, ill-conditioned (record 3514739)</i>							
case6024	3012wp x2	6,024	595	11,451	72,553	818	fail
case6243	3375wp + 2869pegase	6,243	901	11,583	77,335	1.45e+03	fail
case6748	3375wp x2	6,748	783	12,711	81,461	1.45e+03	fail
case7092	2869pegase x2 + 1354pegase	7,092	1,279	12,903	89,043	562	fail
case9961	2869pegase x3 + 1354pegase	9,961	1,789	18,131	125,663	562	fail
case10595	9241pegase + 1354pegase	10,595	1,704	19,484	145,244	562	fail
case12110	9241pegase + 2869pegase	12,110	1,954	22,264	166,032	559	fail
<i>Added by us, loading limit (record 3491654)</i>							
case14limit		14	4	22	146	3.75	fail
case_ieee30limit		30	5	53	379	2.77	10
case57limit		57	6	106	718	3.81	10
case89pegaselimit		89	11	165	1,811	324	fail
case118limit		118	53	181	1,051	5.89	10
case300limit		300	68	530	3,736	25.8	10
case1354pegaselimit		1,354	259	2,447	15,803	562	fail
case2383wplimit		2,383	326	4,438	27,874	9.79	10
case2736splimit		2,736	233	5,237	33,715	438	fail
case2737soplmit		2,737	192	5,280	34,242	761	fail
case2746woplmit		2,746	349	5,141	32,445	512	fail
case2746wplimit		2,746	363	5,127	32,129	513	fail
case2869pegaselimit		2,869	509	5,227	36,591	559	10
case3012wplimit		3,012	297	5,725	36,263	818	fail
case3120splimit		3,120	247	5,991	38,329	611	fail
case3375wplimit		3,374	391	6,355	40,717	1.45e+03	fail
case9241pegaselimit		9,241	1,444	17,036	129,412	532	fail
case13659pegaselimit		13,659	4,091	23,225	174,703	201	8

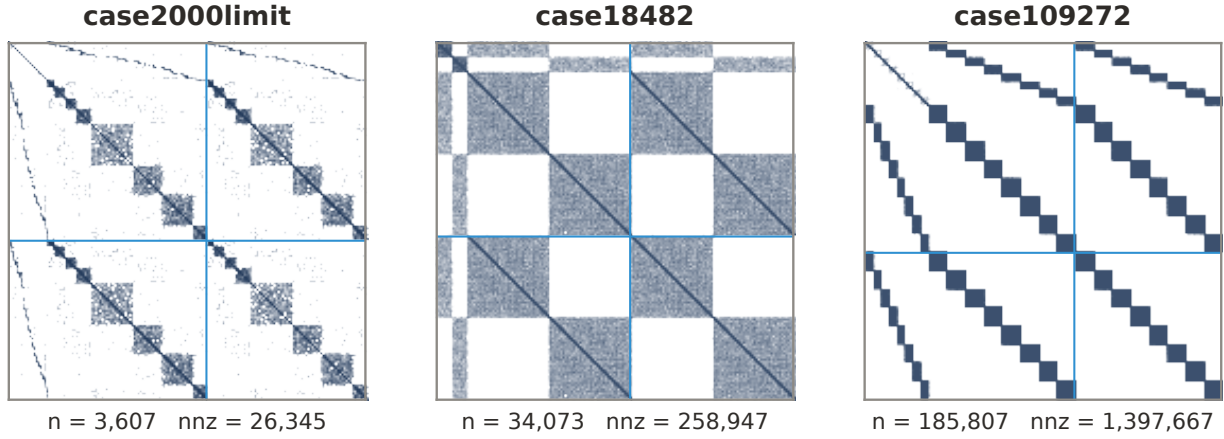


Figure 3. Nonzero pattern of the Jacobian at the flat start. The orange lines separate the angle unknowns from the voltage-magnitude unknowns, so each matrix shows the four blocks $\partial P/\partial\theta$, $\partial P/\partial V$, $\partial Q/\partial\theta$, $\partial Q/\partial V$. The ill-conditioned systems are copies of one network placed on the diagonal.

The flat start includes the slack angle. All angles, including the slack’s, start at zero. For “NR-MAT” (NR from the operating point stored in the file) we shift the stored angles so the slack is at zero; both conventions lead to the same solution.

Many limit cases sit exactly on NR’s budget. Six of the 22 limit cases converge from the flat start in *exactly* ten NR iterations, MATPOWER’s default limit, and eleven more fail at ten. This is how the dataset was built, and it means any change worth one NR iteration can flip a limit case from failure to success. Section 8.6 separates such cases from genuine rescues.

More than one solution. The power flow equations have several roots, and a mismatch below 10^{-8} does not say which one was found. For every system we take as reference the solution NR reaches from the initial guess stored in the case file (up to 50 iterations, with the optimal multiplier as a fallback on the most stressed cases; `improvements.cases.reference_voltage`). A run counts as *solved* only if every bus voltage is within 10^{-4} pu of the reference. This check changes the modification study substantially (Section 8.1).

4 Methods Implemented from the Base Paper

4.1 Software architecture

Everything in `dynhomotopy` is written against one small interface: a problem supplies $g(x)$ and $J(x)$, and nothing else. The paper’s 2×2 tutorial problem and the 109,272-bus grid therefore go through exactly the same solver and integrator code. Figure 4 shows the layers. Our modifications live in a second package that imports the first and never edits it, so the reproduction cannot be disturbed by later work; a test asserts that with every switch off, the modified solver returns the paper’s method bit for bit.

4.2 The homotopy and its three integrators

The class `FixedPointHomotopy` implements (5) and its derivatives in a few lines, and each integration scheme is a function from $(x_k, t_k, \Delta t)$ to x_{k+1} . The listing below is the actual code,

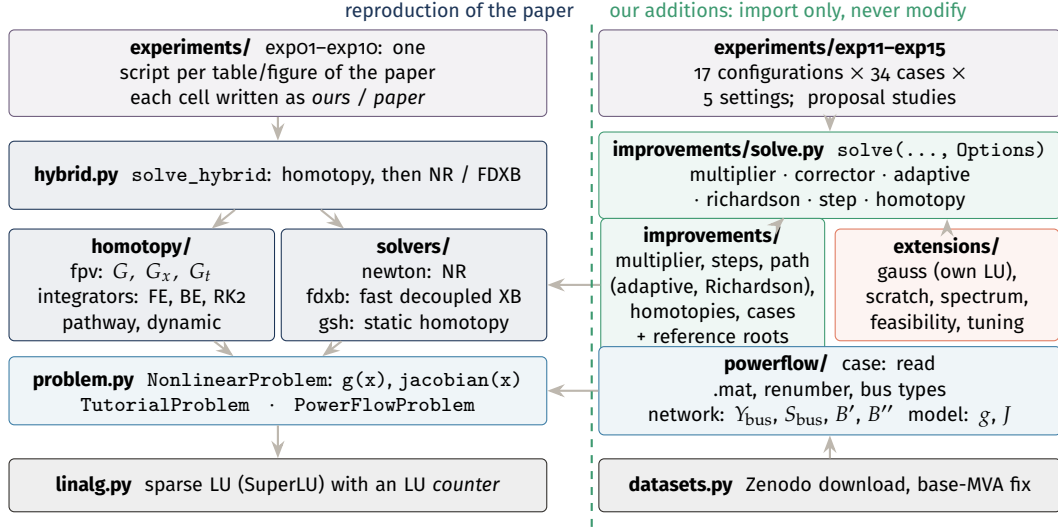


Figure 4. Architecture of the implementation. Every solver sees only $g(x)$ and $J(x)$; every LU factorization in the whole program goes through one counter, which is how cost is measured in Section 8.

lightly trimmed (Listing 1). With $\Phi(x, t) = -G_x(x, t)^{-1}G_t(x)$ the three schemes are

$$\text{FE: } x_{k+1} = x_k + \Delta t \Phi(x_k, t_k), \quad (6)$$

$$\text{RK2: } k_1 = \Phi(x_k, t_k), \quad k_2 = \Phi(x_k + \Delta t k_1, t_{k+1}), \quad x_{k+1} = x_k + \frac{\Delta t}{2}(k_1 + k_2), \quad (7)$$

$$\text{BE: } x_{k+1} = x_k + \Delta t \Phi(\hat{x}_{k+1}, t_{k+1}), \quad \hat{x}_{k+1} = x_k \text{ (one fixed-point iteration)}. \quad (8)$$

The implicit equation of true backward Euler, $x_{k+1} = x_k + \Delta t \Phi(x_{k+1}, t_{k+1})$, is replaced by one fixed-point iteration started at x_k , as in the paper. What remains “implicit” about BE is only that it evaluates G_x at the *new* time t_{k+1} . That single detail is the whole story of the next subsection.

Listing 1 The core of the method, from `homotopy/fpv.py` and `homotopy/integrators.py`

```

1 class FixedPointHomotopy:                                # G(x, t) = t g(x) + (1-t) K (x - x0)
2     def G(self, x, t):
3         return t * self.problem.g(x) + (1 - t) * self.K * (x - self.x0)
4     def Gx(self, x, t):
5         return (t * self.problem.jacobian(x) + (1 - t) * self.K * self.eye).tocsc()
6     def Gt(self, x):
7         return self.problem.g(x) - self.K * (x - self.x0)
8     def dxdt(self, x, t):                                  # one sparse LU per call, counted
9         return -solve(self.Gx(x, t), self.Gt(x), self.counter)
10
11 def forward_euler(h, x, t, dt):    return x + dt * h.dxdt(x, t)
12 def backward_euler(h, x, t, dt):  return x + dt * h.dxdt(x, t + dt) # Gx at the NEW time
13 def runge_kutta2(h, x, t, dt):
14     k1 = h.dxdt(x, t); k2 = h.dxdt(x + dt * k1, t + dt)
15     return x + dt / 2 * (k1 + k2)

```

4.3 What the first step really does

At $t_0 = 0$ the state is $x^{(0)}$, so $G_t = g(x^{(0)})$ and the two schemes differ only in where they evaluate G_x .

Forward Euler. uses $G_x(x^{(0)}, 0) = KI$: the network is invisible. The step is

$$\Delta x_{\text{FE}} = -\frac{\Delta t_0}{K} g(x^{(0)}) = -50 g(x^{(0)}) \quad (K = 10^{-4}, \Delta t_0 = 0.005). \quad (9)$$

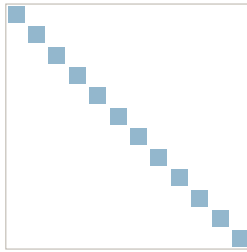
Every unknown is pushed by fifty times its own mismatch, and on the ill-conditioned systems the mismatch is in the hundreds, so angles move by thousands of radians.

Backward Euler. uses $G_x(x^{(0)}, \Delta t_0) = \Delta t_0 J + (1 - \Delta t_0)KI$, and the step becomes

$$\begin{aligned} \Delta x_{\text{BE}} &= -\Delta t_0 [\Delta t_0 J + (1 - \Delta t_0)KI]^{-1} g(x^{(0)}) = -[J + \delta' I]^{-1} g(x^{(0)}), \\ \delta' &= \frac{(1 - \Delta t_0)K}{\Delta t_0} \approx \frac{K}{\Delta t_0} = \delta = 0.02. \end{aligned} \quad (10)$$

This is a Newton step on g with the Jacobian shifted by δI —structurally a Levenberg–Marquardt step [17, 18]. The paper derives the same expression (its eq. 25) and calls δ a perturbation. The shift does two things. It keeps the linear system solvable where the flat-start Jacobian is nearly singular, and it shortens the step in the directions where J is weak, which is exactly where a plain Newton step overshoots. The linearised static homotopy (the paper’s eq. 27) gives the same point, and our test suite checks the two agree to 10^{-10} .

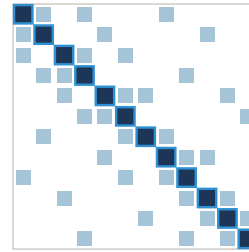
Forward Euler sees $G_x = KI$



only a diagonal of 10^{-4} :
no network information

$$\begin{aligned} \Delta x &= -\frac{\Delta t_0}{K} g(x^{(0)}) \\ &= -50 g(x^{(0)}): \text{blows up} \end{aligned}$$

Backward Euler sees $G_x \propto J + \delta I$



full network Jacobian,
diagonal lifted by $\delta = K/\Delta t_0$

$$\begin{aligned} \Delta x &= -[J + \delta I]^{-1} g(x^{(0)}) \\ &\text{a damped Newton step} \end{aligned}$$

Figure 5. What each integrator “sees” on the first step. The matrix that forward Euler inverts at $t = 0$ is KI , so its step ignores the network entirely; backward Euler evaluates G_x one step later, where it is proportional to the real Jacobian plus a small diagonal shift.

The last step is almost Newton.. The paper’s favourite path has only one more point, $t = 1$. With BE, G_x is evaluated at $t = 1$, where $G_x = J(x_1)$ and K disappears from it:

$$x(1) = x_1 - (1 - \Delta t_0) J(x_1)^{-1} [g(x_1) - K(x_1 - x^{(0)})] \approx x_1 - J(x_1)^{-1} g(x_1). \quad (11)$$

The factor 0.995 and the term $K(x_1 - x^{(0)})$, with $K = 10^{-4}$, are tiny. So the two-point BE “homotopy” is one shifted Newton step followed by an almost plain Newton step, after which NR proper takes over. This explains three things the paper reports. Its cost is essentially that of NR (two LUs for the path, then 3–4 for NR). The regularising work happens entirely in the first step, which is why δ , not K or Δt_0 alone, is the parameter that matters. And explicit schemes cannot compete with a large final step, because at $t = 1$ they would have to use G_x evaluated at the previous, smaller t .

4.4 The refining solvers

NR. (solvers/newton.py) stops when $\|g\|_\infty < 10^{-8}$ or after 10 iterations, MATPOWER’s defaults and the paper’s setting. A singular Jacobian or a non-finite mismatch ends the run as a failure.

FDXB. (solvers/fdxb.py) follows MATPOWER’s fdpf: B' ignores shunts, line charging, taps and resistance; B'' keeps the full network without phase shifters; both are factorised once. Each iteration is a P – θ update followed by a Q – V update. We report the number of P updates plus Q updates, because that is what the paper’s Table 7 counts (for example $15 + 14 = 29$); this is not fdpf’s return value, which counts P/Q cycles.

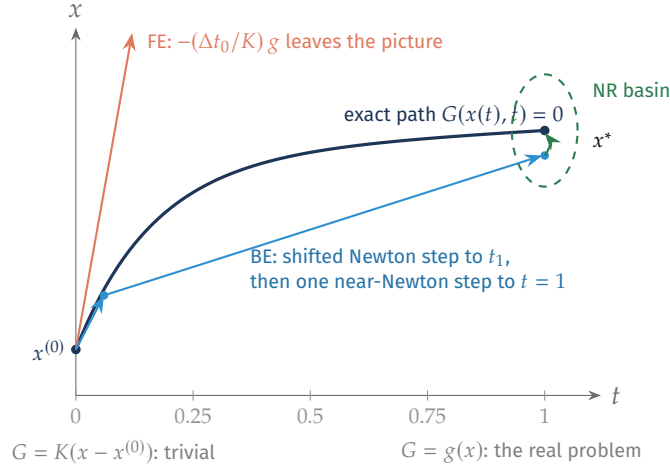


Figure 6. Schematic of the dynamic homotopy. BE does not track the path closely—it only needs to land inside the region from which NR converges. A real two-dimensional instance of this picture, computed with our code, is Figure 7.

GSH-NR. (`solvers/gsh.py`) is the paper’s static-homotopy baseline from [3]. The paper describes it only in words, so ours is a reconstruction: fictitious shunts sized to make the flat start an exact solution at $h = 0$ and scaled by $(1 - h)$, and the impedance of branches at the slack bus multiplied by $\delta + (1 - \delta)h$; h runs from Δh to 1 and every point is a full NR solve. Where our version and the paper’s differ is recorded in Section 6.

Algorithm 1 The base paper’s hybrid method (as implemented in `hybrid.py`)

Input: problem (g, J) , flat start $x^{(0)}$, factor K , time points $0 = t_0 < t_1 < \dots < t_N = 1$, scheme $\in \{\text{FE}, \text{RK2}, \text{BE}\}$

- 1: $x \leftarrow x^{(0)}$
- 2: **for** $k = 0, \dots, N - 1$ **do**
- 3: $\Delta t \leftarrow t_{k+1} - t_k$; **if** $k = 0$ **then** use BE **else** use the chosen scheme
- 4: $x \leftarrow \text{STEP}(x, t_k, \Delta t)$ ▷ each Φ evaluation = one sparse LU
- 5: **if** a matrix is singular **or** $\|g(x)\|_\infty$ is not finite **then return failure**
- 6: **end if**
- 7: **end for**
- 8: **return** NR(x) or FDXB(x) ▷ the refiner starts from $\hat{x}^{(0)} = x(1)$

4.5 Verification before measurement

Before any number was compared with the paper, the building blocks were tested on their own (`tests/`, 24 tests, all passing):

- the analytic Jacobian agrees with central finite differences on two real systems;
- the path generator produces the paper’s time grids, including the shortened last step;
- the BE first step and the linearised static homotopy (the paper’s eqs. 25 and 27) give the same point;
- NR on the tutorial takes exactly 11 iterations to the root $(1.5, 0.5)$ from $x_2^{(0)} = 0.995$, as in the paper’s Fig. 1(a);
- NR-MAT and NR-flat iteration counts, one full row of Table 4 and the FDXB count of Table 7 for `case2000limit` match the paper;

- with every modification switched off, the modified solver returns the paper’s method exactly;
- our Gauss elimination and LU agree with NumPy, pivot past a zero diagonal, and give the same Newton iterates as SuperLU on case500limit;
- the power method and shifted inverse iteration find the known eigenvalues of a diagonal matrix, and golden-section search finds the minimum of a parabola;
- Richardson step control reaches $t = 1$ and converges, and both command-line tools run.

5 Reproducing the Base Paper

Each of the paper’s results has its own script in `experiments/` (Table 11 in the appendix). Every table cell below is shown as our value above the paper’s value.

5.1 The two-variable tutorial

The paper first tests the idea on $g_1 = x_1^2 + x_2^2 - 2x_1x_2 - 1$, $g_2 = x_1 + x_2 - 2$ from $x^{(0)} = (1, 1)$, where the Jacobian is singular. The system has two roots, $A = (1.5, 0.5)$ and $B = (0.5, 1.5)$, and the singular set is the whole line $x_1 = x_2$ through the starting point. Figure 7 draws what the three integrators actually do in this plane, which the paper only shows as mismatch curves. The exact homotopy path (traced with 800 corrected steps) bends from $x^{(0)}$ to root A. With $K = 0.05$ all three coarse integrators stay near it; with $K = 0.005$ the explicit ones overshoot across the singular line and NR then converges to root B, while BE stays on the right side and reaches A.

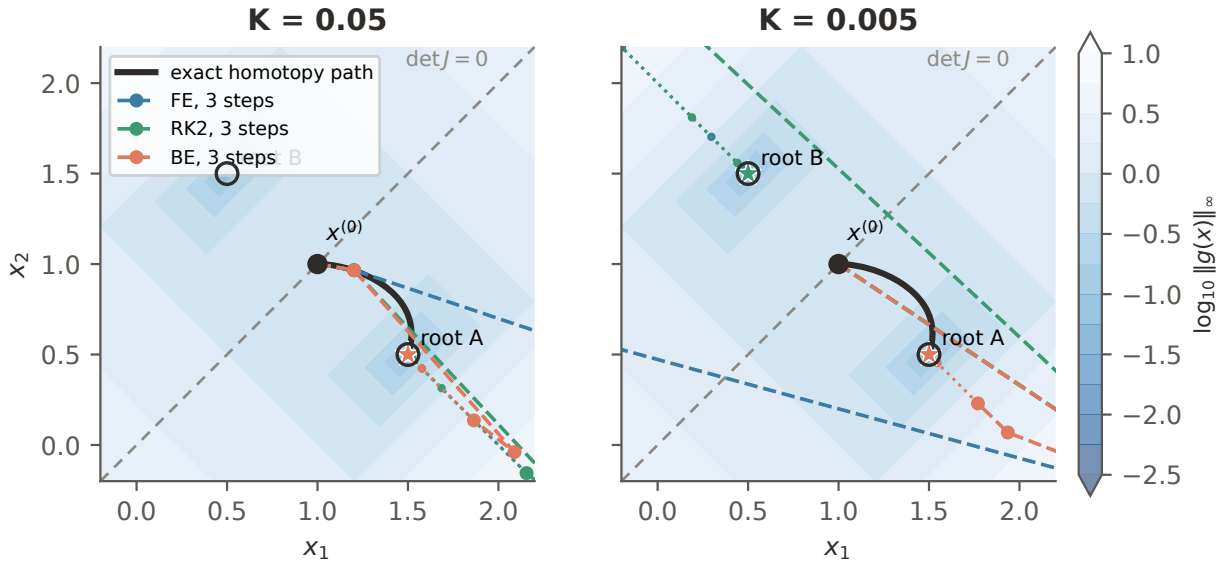
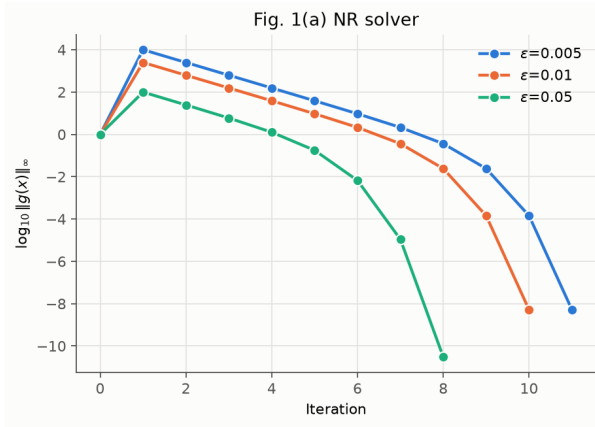
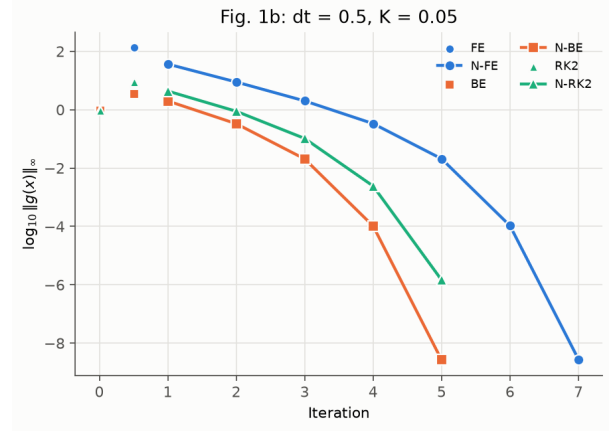
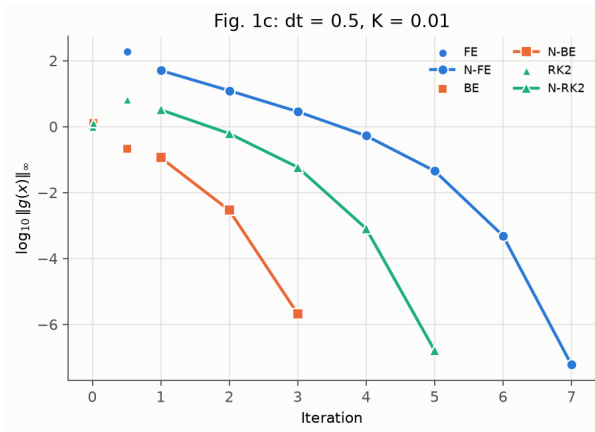
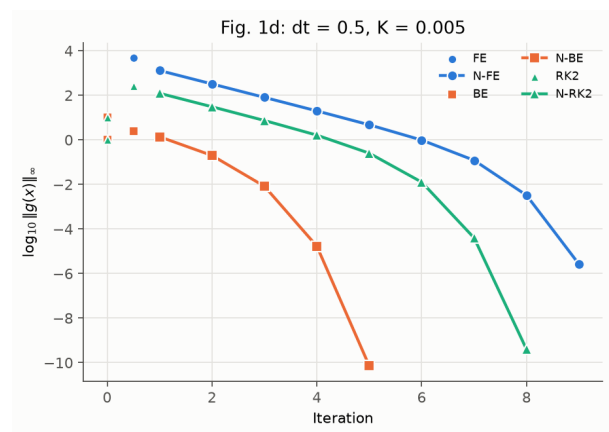
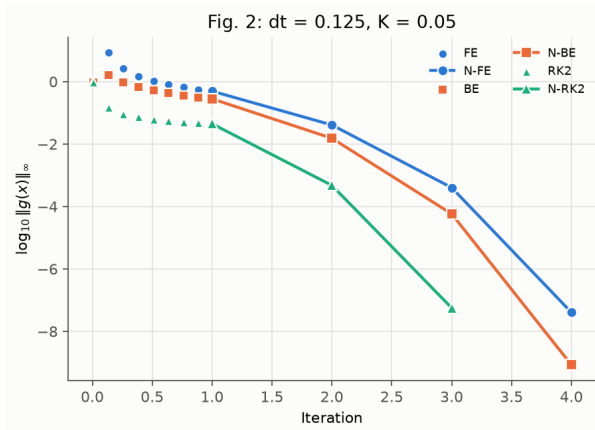


Figure 7. The tutorial problem in the (x_1, x_2) plane, $\Delta t_0 = 0.01$ then $\Delta t = 0.5$. Background: $\log_{10} \|g\|_\infty$. Black: the exact homotopy path. Dashed: the three coarse integrations; dotted: the NR iterations that follow; stars: where NR ends. At the smaller K the explicit schemes cross the singular line $\det J = 0$ and NR finds the other root.

Figure 8 and Table 2 compare iteration counts. NR alone matches the paper for $\epsilon = 0.005$ and 0.01 (11 and 10 iterations); for $\epsilon = 0.05$ we need 8, not 9, and we show in Section 6 that the paper’s curve for that case cannot come from these equations. In the hybrid runs BE is the best start for NR in every panel, as in the paper, but some counts differ by one or two.

(a) Fig. 1(a): NR alone from $(1, 1 - \epsilon)$ (b) Fig. 1(b): homotopy then NR, $\Delta t = 0.5, K = 0.05$ (c) Fig. 1(c): $\Delta t = 0.5, K = 0.01$ (d) Fig. 1(d): $\Delta t = 0.5, K = 0.005$ (e) Fig. 2: smaller step $\Delta t = 0.125, K = 0.05$

Reading the panels. Scattered markers are the homotopy points, drawn at their time $t \in [0, 1]$; the connected curves are the NR iterations that follow, iteration i drawn at $1 + i$. Orange squares (BE) reach 10^{-5} first in every hybrid panel, as in the paper. In (c) and (d) the FE and RK2 runs converge, but to root B rather than A (Figure 7). With the smaller step of panel (e) all three schemes give NR a good start.

Figure 8. Our versions of all five tutorial panels of the paper (Figures 1(a)–(d) and 2), generated by `exp01_tutorial.py`.

Table 2. Tutorial: NR iterations to $\|g\|_\infty < 10^{-5}$, ours and read off the paper’s plots, and the root reached.

Run	Δt	K	Ours	Paper	Root
NR only, $\epsilon = 0.005$	–	–	11	11	A
NR only, $\epsilon = 0.01$	–	–	10	10	A
NR only, $\epsilon = 0.05$	–	–	8	9	A
FE / BE / RK2 then NR	0.5	0.05	6 / 4 / 4	6 / 3 / 6	A / A / A
FE / BE / RK2 then NR	0.5	0.01	6 / 2 / 4	4 / 2 / 4	B / A / B
FE / BE / RK2 then NR	0.5	0.005	8 / 4 / 7	6 / 3 / 7	B / A / B
FE / BE / RK2 then NR	0.125	0.05	3 / 3 / 2	3 / 2 / 3	A / A / A

5.2 The first time step

Section 4.2.1 of the paper argues that explicit schemes cannot take the first step. Figure 9 measures it on all nine systems: FE multiplies the mismatch by factors between 7×10^3 and 1.8×10^{10} ; RK2, which evaluates a second slope at the FE point, does no better; BE reduces the mismatch by roughly a factor of eight on every ill-conditioned system (533 to 65, 201 to 25), exactly the paper’s numbers. Taking the explicit step with $K = 1$ instead, the value earlier work used, avoids the explosion but moves the state so little that nothing is gained.

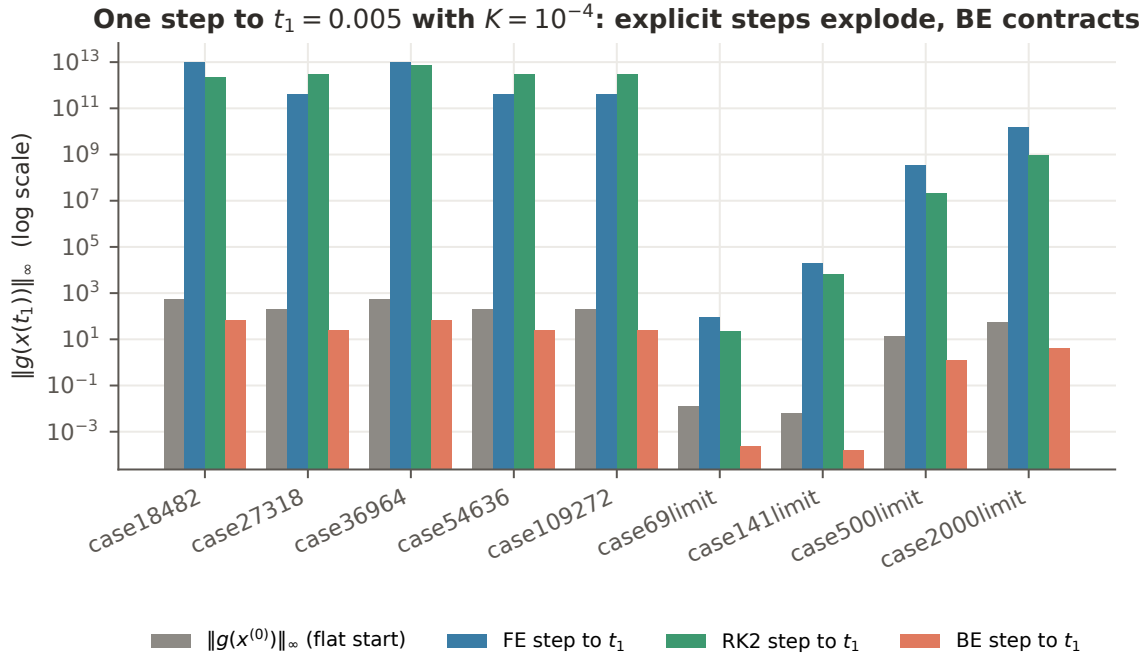


Figure 9. Mismatch after the single first step to $t_1 = 0.005$, $K = 10^{-4}$ (log scale). Grey is the flat start. Forward Euler and RK2 make every system worse by orders of magnitude; backward Euler improves every one.

5.3 Tables 2–4: one large step after t_2

The paper’s Tables 2–4 use the path $\{0, 0.005, 0.01, 0.51, 1\}$ with $K = 10^{-4}$: BE to t_1 , then the scheme under test with a large step. Table 3 shows the BE results: they match the paper almost digit for digit, including the three NR iterations that follow. For case18482 we get 533, 65, 33, 7.2, 3.7 along the path and $0.042, 1.5 \times 10^{-5}, 7.7 \times 10^{-12}$ in NR, against the paper’s 532, 65, 33, 7.2, 3.7 and $0.04, 1.5 \times 10^{-5}, 10^{-11}$. The FE and RK2 tables (Section B) match wherever the schemes

stay finite; where they blow up, they blow up at the same point but not to the same size, which is expected once a number is 10^9 or more and is dominated by rounding.

Table 3. Table 4 re-run: $\|g\|_\infty$ along the BE path $\{0, 0.005, 0.01, 0.51, 1\}$ and after each of the first three NR iterations, $K = 10^{-4}$. Each case shows our value above the paper’s.

Case		$t=0$	0.005	0.01	0.51	1	NR 1	NR 2	NR 3	Result
case18482	ours	533	65	33	7.2	3.7	0.042	1.5×10^{-5}	7.7×10^{-12}	conv. (3)
	paper	532	65	33	7.2	3.7	0.04	1.5×10^{-5}	10^{-11}	
case27318	ours	201	25	13	0.5	0.25	0.096	2.5×10^{-5}	7.4×10^{-12}	conv. (3)
	paper	200	25	13	0.5	0.25	0.1	2.5×10^{-5}	7×10^{-12}	
case36964	ours	533	66	33	6.9	3.6	0.27	6.5×10^{-4}	4.5×10^{-9}	conv. (3)
	paper	532	65	33	6.9	3.6	0.27	6×10^{-4}	4×10^{-9}	
case54636	ours	201	25	13	0.5	0.25	0.096	2.5×10^{-5}	7.4×10^{-12}	conv. (3)
	paper	200	25	13	0.5	0.25	0.1	3×10^{-5}	7×10^{-12}	
case109272	ours	201	25	13	0.5	0.25	0.096	2.5×10^{-5}	7.4×10^{-12}	conv. (3)
	paper	200	25	13	0.5	0.25	0.1	3×10^{-5}	7×10^{-12}	
case69limit	ours	0.012	2.4×10^{-4}	1.2×10^{-4}	1.6×10^{-6}	4.6×10^{-7}	3.9×10^{-12}	–	–	conv. (1)
	paper	0.012	2×10^{-4}	10^{-4}	2×10^{-6}	5×10^{-7}	5×10^{-12}	–	–	
case141limit	ours	6.4×10^{-3}	1.6×10^{-4}	8.2×10^{-5}	10^{-6}	4.5×10^{-7}	1.2×10^{-11}	–	–	conv. (1)
	paper	6×10^{-3}	2×10^{-4}	8×10^{-5}	10^{-6}	5×10^{-7}	6×10^{-11}	–	–	
case500limit	ours	13	1.2	0.62	0.017	8.7×10^{-3}	1.1×10^{-6}	3.5×10^{-13}	–	conv. (2)
	paper	13	1.2	0.6	0.02	9×10^{-3}	10^{-6}	2×10^{-13}	–	
case2000limit	ours	55	3.9	2	2.8	1.5	0.01	7.5×10^{-7}	1.1×10^{-12}	conv. (3)
	paper	54	3.9	2	2.8	1.5	0.01	8×10^{-7}	9×10^{-13}	

5.4 Table 5: growing steps

Table 5 of the paper uses, on three systems, a path whose steps grow geometrically:

$$\{0, 0.005, 0.01, 0.02, 0.12, 0.62, 1\}.$$

With more points, all three schemes give NR a usable start, although after FE the NR stage needs five iterations instead of three on both case36964 and case2000limit. All of our cells that carry a number (89 of the 90; one is blank in both) agree with the paper’s to within a factor of two (Table 4 and Figure 10).

5.5 How close is the reproduction, overall?

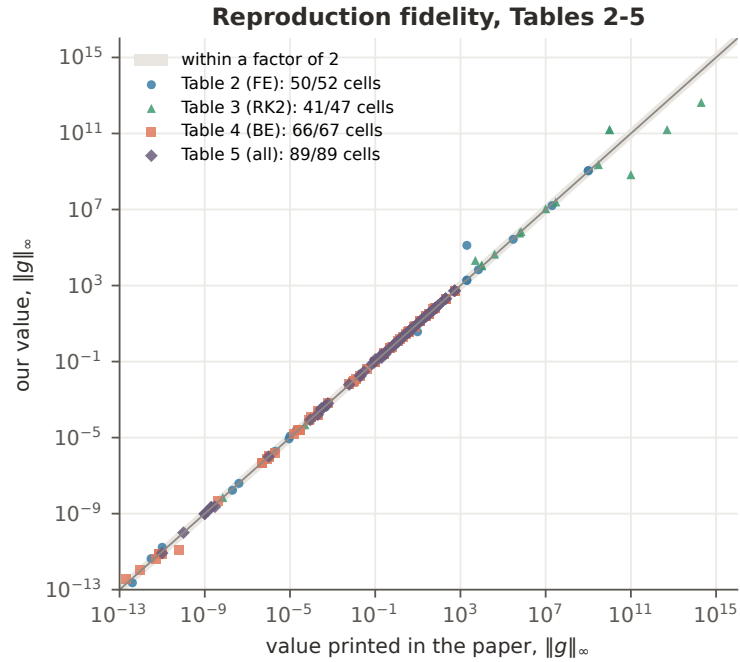
Figure 10 puts every numeric cell of Tables 2–5 on one plot: 255 pairs of (paper, ours). 246 of them (96%) agree within a factor of two, and 199 (78%) within 10%, which is the rounding of the paper’s two printed digits. Of the nine outliers, seven are divergent FE and RK2 values above 10^3 , one is the Table 2 cell discussed in Section 6, and one is a final NR mismatch near 10^{-11} , at rounding level.

5.6 Section 4.4: larger first steps

Section 4.4 of the paper raises Δt_0 to 0.05 and K to 0.001, keeping $\delta = 0.02$, and tries paths of three, four and five points. It reports that with few points only BE works, and that raising Δt_0 further to 0.1 (with $K = 0.002$) fails. Figure 11 is our full grid. BE gives NR a usable start in 75 of 81 (path, system) cells, FE in 64 and RK2 in 59. On three-point paths BE works on 8 of 9 systems and FE and RK2 on only 3, confirming the paper’s main claim. Two details disagree: FE and RK2 also succeed on several four-point paths, and BE followed by NR fails on case36964 with only three points (FDXB still converges). And for $\Delta t_0 = 0.1$ the picture depends on the path,

Table 4. Table 5 re-run, $K = 10^{-4}$: $\|g\|_\infty$ at each path point and after three NR iterations.

Case	Solver		$t=0$	0.005	0.01	0.02	0.12	0.62	1	NR 1	NR 2	NR 3
case36964	FE	ours	533	66	3.5	0.72	5.7	57	114	28	4.7	0.23
		paper	532	65	3.5	0.7	5.7	57	114	28	5	0.2
	RK2	ours	533	66	33	16	9.6	4.2	2.6	0.19	3×10^{-4}	9.8×10^{-10}
		paper	532	65	33	16	9.6	4.2	2.6	0.2	3×10^{-4}	10^{-9}
	BE	ours	533	66	33	17	4	1.4	0.91	0.091	8.7×10^{-5}	10^{-10}
		paper	532	65	33	16	4	1.4	0.9	0.09	9×10^{-5}	10^{-10}
case109272	FE	ours	201	25	1.2	0.019	0.074	0.29	0.17	0.03	6.3×10^{-4}	2.1×10^{-9}
		paper	200	25	1.2	0.02	0.07	0.3	0.17	0.03	6×10^{-4}	2×10^{-9}
	RK2	ours	201	25	13	6.3	1.2	0.51	0.31	0.19	1.6×10^{-4}	2.2×10^{-9}
		paper	200	25	13	6.3	1.2	0.5	0.3	0.19	2×10^{-4}	2×10^{-9}
	BE	ours	201	25	13	6.3	1.1	0.21	0.13	0.017	10^{-6}	8.4×10^{-12}
		paper	200	25	13	6.3	1.1	0.2	0.1	0.02	10^{-6}	10^{-11}
case2000limit	FE	ours	55	3.9	0.78	0.66	3.6	58	91	16	1	6.2×10^{-3}
		paper	55	3.9	0.8	0.7	3.6	58	91	16	1	6×10^{-3}
	RK2	ours	55	3.9	1.9	0.97	5	5.5	3.4	0.21	3.5×10^{-4}	9.8×10^{-10}
		paper	55	3.9	1.9	1	5	5.5	3.4	0.2	3×10^{-4}	10^{-9}
	BE	ours	55	3.9	2	1	0.98	0.25	0.16	4.8×10^{-4}	2.3×10^{-9}	–
		paper	55	3.9	1.9	1	1	0.3	0.2	5×10^{-4}	3×10^{-9}	–

**Figure 10.** Every numeric cell of the paper’s Tables 2–5 against our value, on log axes spanning 28 orders of magnitude. The grey band is agreement within a factor of two.

which the paper does not give: the three-point path works on eight systems, the five-point path $\{0, 0.1, 0.2, 0.3, 1\}$ fails with NR on the four largest.

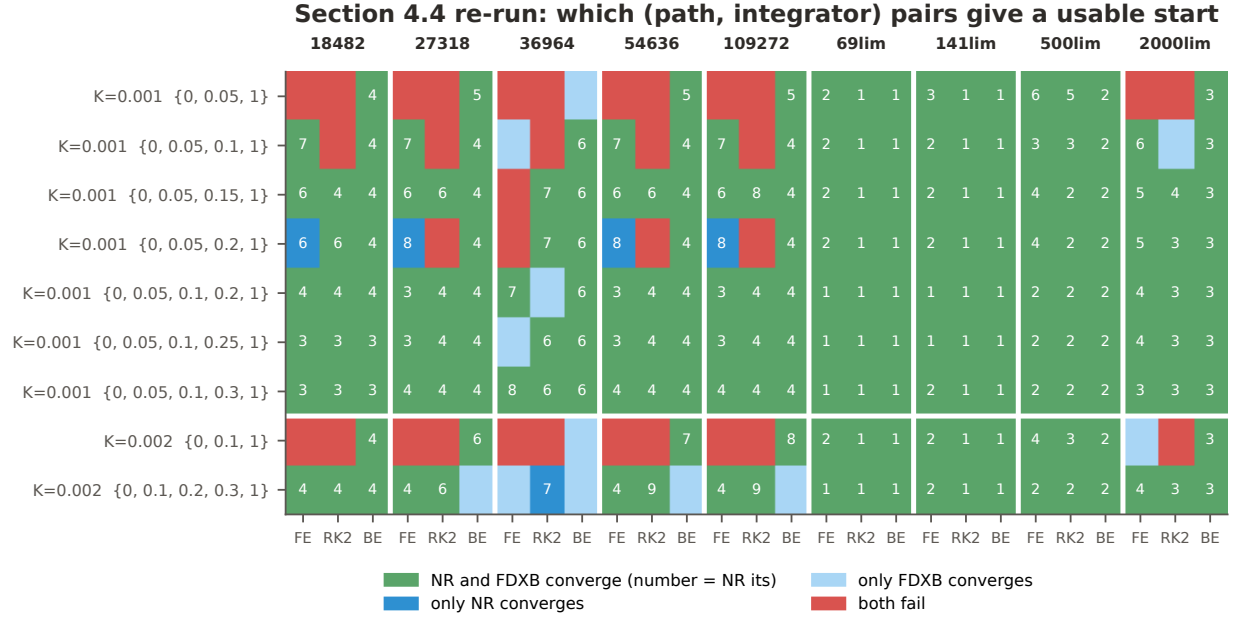


Figure 11. Section 4.4 re-run. Rows: paths and K ; columns: system and scheme. Colour: whether NR and FDXB converge afterwards; number: NR iterations.

5.7 Figure 3 and Table 6: five-point paths

With five-point paths $\{0, 0.05, 0.1, t_3, 1\}$ and $K = 0.001$, Figure 12 reproduces the paper’s Figure 3, including its one failure: FE with $t_3 = 0.25$ on case36964. We find one more: RK2 with $t_3 = 0.20$ on the same system diverges in our run (the dotted green line leaving panel a), where the paper reports a successful solve at 232% of the NR-MAT time.

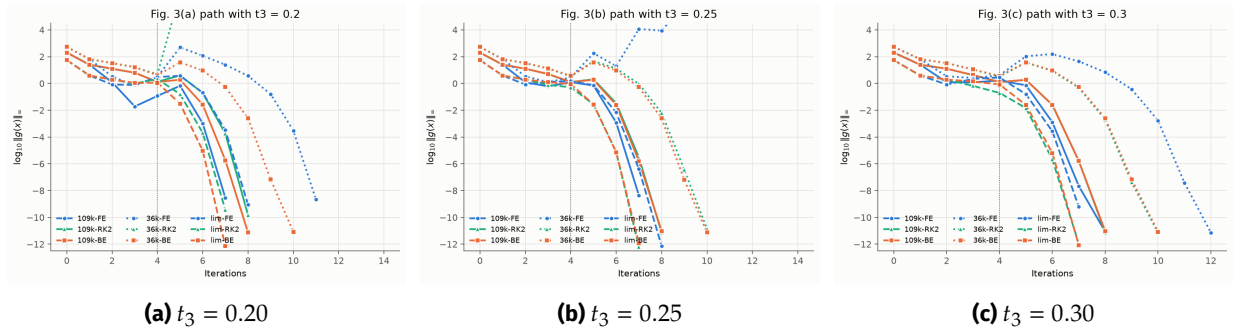


Figure 12. Figure 3 re-run: $\log_{10}\|g\|_{\infty}$ over the five homotopy points (indices 0–4) and the NR iterations that follow, for case109272 (solid), case36964 (dotted) and case2000limit (dashed).

Table 6 times these runs (median of seven). On case109272 our percentages of the NR-MAT time are within 13 percentage points of the paper’s in every row and within 8 in most (FE 143–164% against 145–170%, RK2 224–225% against 226–237%, BE 163–164% against 169–171%). On case36964 they are close for FE and 20–30 points higher for RK2 and BE. On case2000limit they are not: we measure 237–342%, the paper 61–94%. We cannot explain a hybrid run being *faster* than NR-MAT there, because it does strictly more work: four homotopy LUs plus three NR iterations, against NR-MAT’s three iterations. The full table is Table 14 in the appendix.

5.8 Figures 4 and 5: the states themselves

The paper’s Figure 4 follows five bus voltages of case109272 through a two-point BE path and NR. Ours (Figure 13) shows the same shape: angles overshoot to about 60 degrees at $t = 1$ and NR pulls them back within one iteration; bus 6 ends at 1.0045 pu and 21.10 degrees. The mismatch goes $201 \rightarrow 25 \rightarrow 1.26$ along the path and $1.70 \rightarrow 1.7 \times 10^{-2} \rightarrow 7.3 \times 10^{-7} \rightarrow 8.8 \times 10^{-12}$ in NR: four NR iterations, as the paper reports.

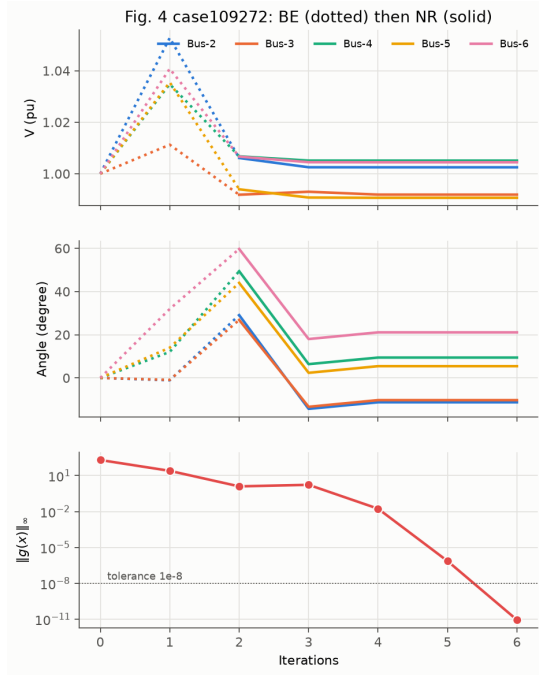


Figure 13. Figure 4 re-run: voltage magnitudes and angles of buses 2–6 of case109272 along the two-point BE path (dotted, points 0–2) and NR (solid). The paper draws the mismatch on a second axis; we give it its own panel. The large angle excursion at $t = 1$ is harmless: the state is already inside NR’s region of convergence.

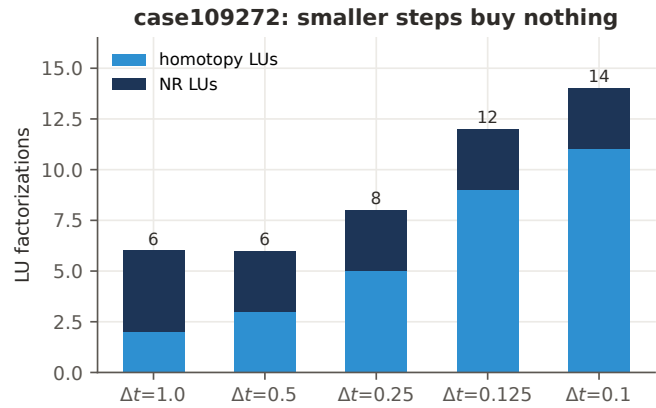


Figure 14. Figure 5 re-run in cost terms: LU factorizations for each constant step Δt after t_1 . Every run ends at the same state (bus 6: 1.0045 pu, 21.10°).

Figure 5 of the paper varies the constant step after t_1 ; Figure 15 is our version. Smaller steps give NR a closer start (bus 6 at 28.2 degrees for $\Delta t = 0.1$ against 59.7 for $\Delta t = 1$) and save one NR iteration, but each extra path point costs an LU, so total cost rises from 6 to 14 factorizations (Figure 14). This is the paper’s own conclusion, now in numbers.

5.9 Table 7: iterations

Table 7 is the paper’s headline comparison. Table 5 gives ours. NR from the flat start fails on all five ill-conditioned grids; BE(NR) needs 4 iterations and RK2(NR) 3 on all of them; FDXB counts match to the iteration. 54 of the 63 cells agree exactly. Six of the nine differences are the two small limit cases, whose NR-MAT, NR-flat and GSH-NR counts match the paper only on the stored 10 MVA base (Table 7); the others are our GSH-NR reconstruction on two systems and one FDXB count for case109272, discussed in Section 6. Every converged run in the table was also checked against the reference operating point of Section 3: all of them reach it, so none of these counts is inflated by convergence to another root.

5.10 Table 8: run time

Figure 16 is our Table 8, the median of seven runs relative to NR-MAT. BE(FDXB) is faster than NR-MAT on all five ill-conditioned systems, at 66–93% of its time against the paper’s 57–85%, and BE(NR) costs slightly more than NR-MAT (102–122% against 92–128%). Both conclusions

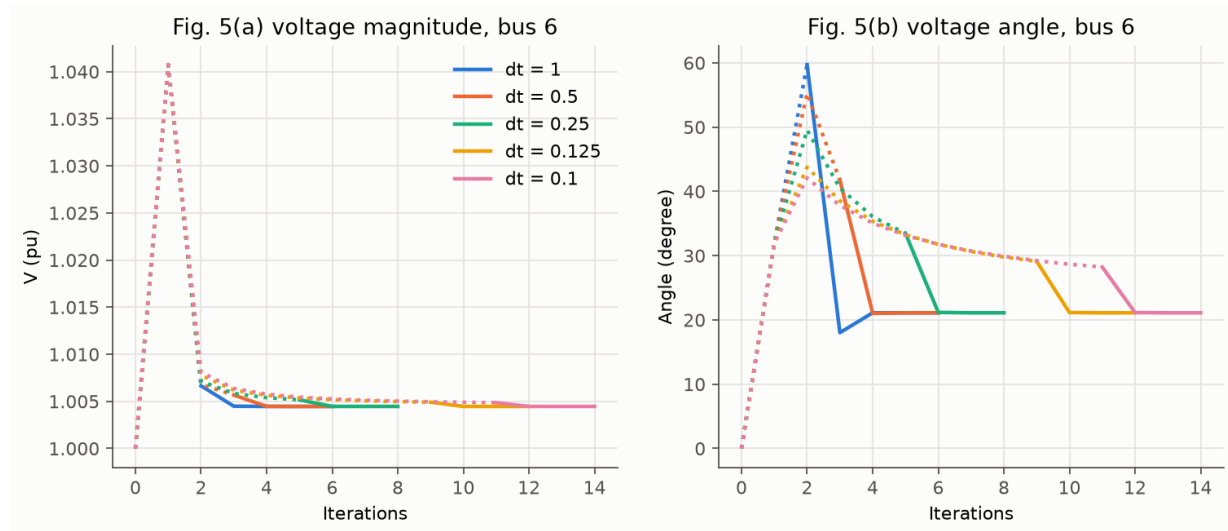


Figure 15. Figure 5 re-run: voltage magnitude (a) and angle (b) of bus 6 of case109272 for five constant steps Δt after t_1 ; dotted is BE, solid is NR. Whatever the step, every run ends at 1.0045 pu and 21.10 degrees.

Table 5. Table 7 re-run: iterations of the refining solver (ours / paper). NR-MAT starts from the case file; all others from the flat start. BE uses the path $\{0, 0.005, 1\}$, RK2 the seven-point path of the paper. Mismatches are in **bold**.

Case	NR-MAT	NR-flat	GSH-NR	BE(NR)	RK2(NR)	BE(FDXB)	RK2(FDXB)
case18482	6 / 6	fail / fail	10 / 10	4 / 4	3 / 3	33 / 33	29 / 29
case27318	5 / 5	fail / fail	fail / 25	4 / 4	3 / 3	29 / 29	25 / 25
case36964	6 / 6	fail / fail	28 / 27	4 / 4	3 / 3	33 / 33	29 / 29
case54636	5 / 5	fail / fail	fail / fail	4 / 4	3 / 3	29 / 29	25 / 25
case109272	5 / 5	fail / fail	fail / fail	4 / 4	3 / 3	29 / 19	25 / 25
case69limit	2 / 4	2 / 4	2 / 4	1 / 1	1 / 1	8 / 8	8 / 8
case141limit	2 / 3	2 / 3	2 / 3	1 / 1	1 / 1	8 / 8	6 / 6
case500limit	3 / 3	4 / 4	4 / 4	2 / 2	2 / 2	13 / 13	11 / 11
case2000limit	3 / 3	5 / 5	5 / 5	3 / 3	3 / 3	22 / 22	20 / 20

of the paper hold. RK2 is much slower for us (252–331% with either refiner) than for the paper (84–218%): it needs seven path points and two LUs per point, and in Python the per-LU overhead is not hidden. Our GSH-NR is also far cheaper relative to NR-MAT than the paper’s (176% and 480% against 1150% and 2157%), a reminder that it is a reconstruction. Absolute times depend on the machine and on its load, which on a laptop is not under our control; percentages of NR-MAT measured in the same session are more stable, and LU counts do not depend on the machine at all. That is why the cost comparison of our own modifications uses LU counts.

Table 8 re-run: cost relative to Newton from the case-file start

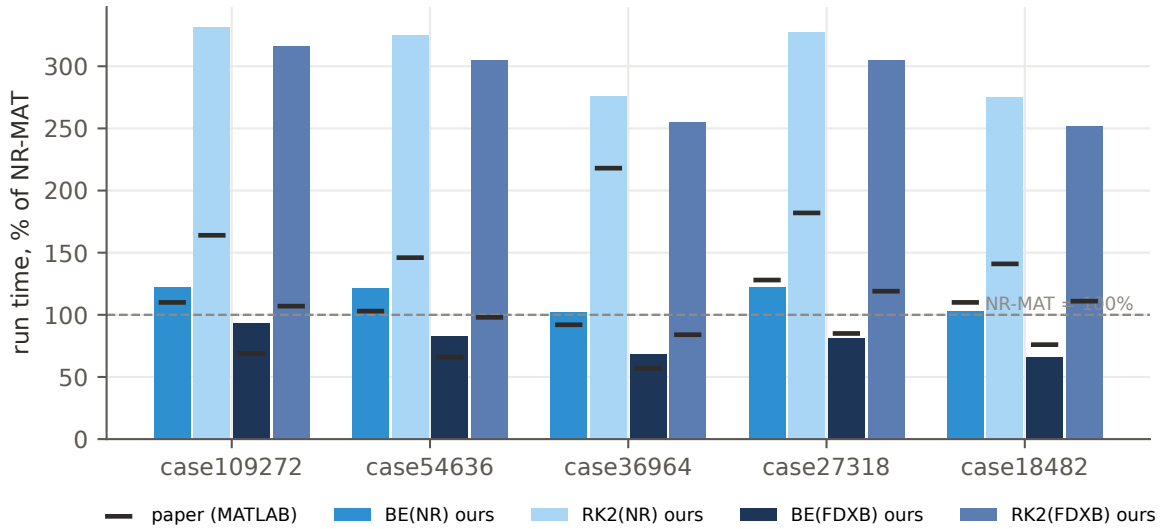


Figure 16. Table 8 re-run: median run time of seven repetitions as a percentage of NR-MAT (bars) against the paper’s MATLAB percentages (black ticks).

What the reproduction establishes. The method works as the paper says. On all five ill-conditioned grids, a two-point backward-Euler path turns a divergent flat start into a four-iteration NR solve, and our numbers track the paper’s across 28 orders of magnitude. Where explicit schemes fail, they fail where the paper says they do, and for the reason Section 4.3 gives.

6 Auditing the Base Paper

A reproduction is only useful if disagreements are chased down rather than averaged away. For every difference we first checked our own code; the eight below survived that check. Table 6 lists them with the evidence and what we did.

Two conventions are not errors but had to be discovered before anything matched: FDXB iterations are counted as P updates plus Q updates (not as MATPOWER’s cycles), and the flat start includes the slack angle. With those fixed, the base-MVA issue is the only one that changes a conclusion, and only for the two smallest systems. The paper does not say whether its converged runs reach the intended operating point; with its own settings all nine of its systems do in our runs (Section 8.1), so this does not affect any of its tables.

Table 6. Eight places where the base paper is inconsistent or under-specified, the evidence, and how our implementation handles each.

Issue	Evidence	Our handling
Base MVA of case69limit and case141limit	The files store 10 MVA. Tables 2–5 match only on 100 MVA; Table 7’s NR-MAT/NR-flat counts (4, 3) match only on 10 MVA, where BE(NR) then needs 2, not 1. No single base reproduces both.	Use 100 MVA; report the 10 MVA counts separately (Table 7).
BE(FDXB) = 19 for case109272 (Table 7)	We get 29, the same as case54636. The two systems are 8 and 4 copies of one network and are identical in every other cell of every table, the paper’s included.	Reported; most likely a typo.
Table 2, FE on case18482 at $t = 0.01$	Paper 9.7, ours 3.7; every other cell of that column matches to two digits.	Reported, unresolved.
Fig. 1(a), $\epsilon = 0.05$	One NR step from (1, 0.95) gives $x = (6.01, -4.01)$ and $\ g\ _\infty \approx 99.5$, i.e. $\log_{10} \approx 2$; the paper’s curve starts near 3. We also need 8 iterations, not 9.	Computed by hand and in code.
GSH-NR specification	Described in words only; no formula for the shunts or for δ . Our reconstruction matches case18482 (10) and nearly case36964 (28 vs 27) but fails on case27318 (paper: 25).	Our version documented in gsh.py.
Section 4.4, $\Delta t_0 = 0.1$	“Does not work” is stated without the path used. With $\{0, 0.1, 1\}$ BE works on 8 of 9 systems; with $\{0, 0.1, 0.2, 0.3, 1\}$ it fails on the four largest with NR.	Both paths reported (Figure 11).
Table 6, RK2 with $t_3 = 0.20$ on case36964	The paper reports success (232%); in our run RK2 diverges on this path.	Reported.
Flat-start mismatch of case2000limit	Tables 2–4 print 54, Table 5 prints 55 for the same starting point (ours: 54.95).	Cosmetic; shows rounding.

Table 7. The two small limit cases on the base stored in the file (ours / paper).

Case, 10 MVA base	NR-MAT	NR-flat	BE(NR)
case69limit	4 / 4	4 / 4	2 / 1
case141limit	3 / 3	3 / 3	2 / 1

7 Our Modifications

7.1 From proposal to implementation

Our course proposal, written before any code, promised RK4 and Richardson-controlled steps for the path, a spectral study of the flat-start Jacobian, maps of the feasible $(K, \Delta t_0)$ region, golden-section tuning and linear solvers written from scratch. All of them are now in the repository (Figure 17); the spectral, feasibility, tuning and linear-solver studies are reported in Section 9. Building the reproduction first also added items the proposal did not foresee: once it was clear (Section 4.3) that the method is a shifted Newton step followed by a near-Newton step, the Newton stages themselves became the most promising place to intervene, which is where the optimal multiplier and the corrector come from. One proposal item changed form: the Richardson estimate uses step doubling rather than an embedded pair, so it works with any of the step rules.

7.2 Design rule: switches on top of an untouched reproduction

Every modification is a field of one immutable Options object in improvements/solve.py. All fields default to the paper’s method, the reproduced dynhomotopy package is imported but never edited, and a test checks that Options() reproduces the paper’s result exactly. This makes each comparison a controlled experiment: exactly one thing differs.

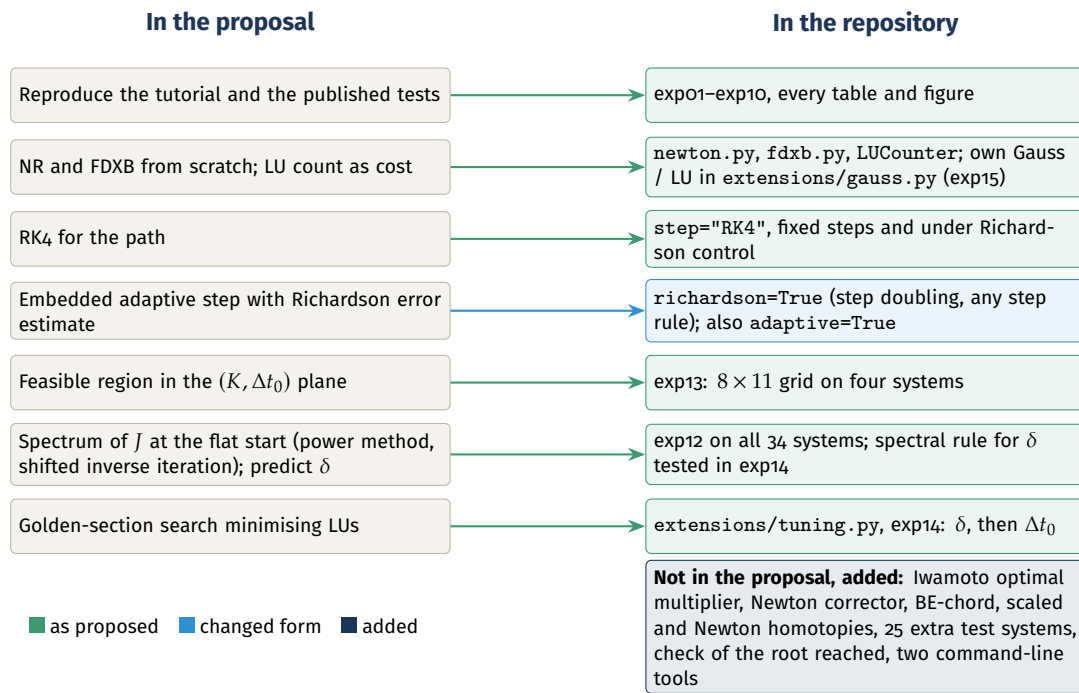


Figure 17. What the proposal promised and what the repository contains.

Listing 2 Using a modification, from Python and from the command line

```

from improvements import Options, solve
result = solve(pf, pf.flat_start(), K=1e-4, times=[0, 0.005, 1],
               options=Options(multiplier=True, corrector=True))

python -m improvements case36964 --corrector --multiplier # printed next to the paper's method
python -m improvements case6024 --richardson --dt0 0.05 --K 1e-3
  
```

Figure 18 shows where each switch acts in the pipeline. The next subsections define them. adaptive and richardson both choose the time points, so they cannot be combined.

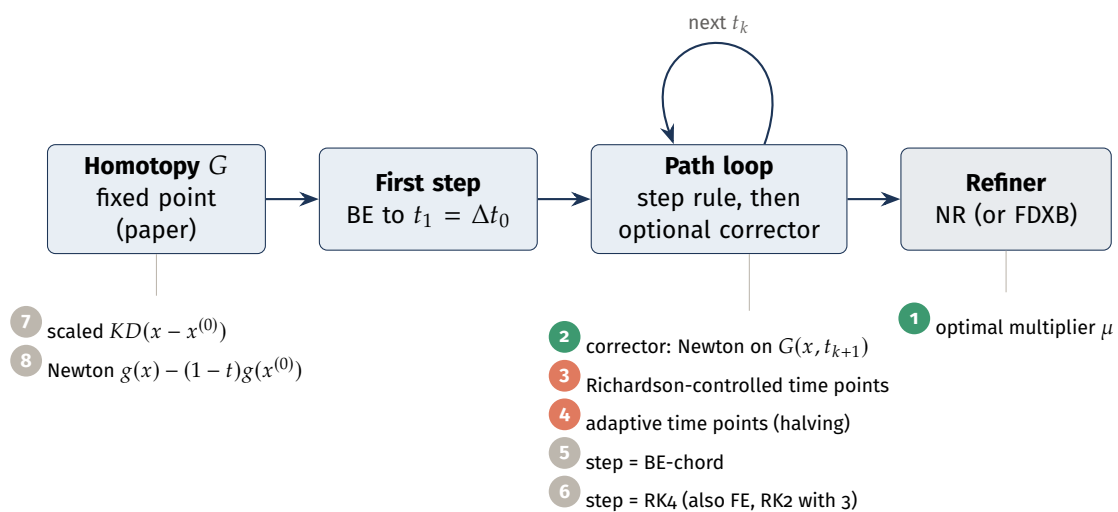


Figure 18. Where each of the eight switches acts. Green badges: the two that help on their own and together (Section 8); orange: the two path-control rules, which help on some systems at a price; grey: the four that did not work.

7.3 Modification 1: the optimal multiplier in NR

After the homotopy, the state is inside NR's region of convergence but not deep inside it; the first NR steps can still overshoot (the angles in Figure 13 jump by 40 degrees). Iwamoto and Tamura [5] scale each Newton step Δx by a factor μ . With $a = g(x)$ and $c = g(x + \Delta x)$, the mismatch along the step is modelled as

$$g(x + \mu \Delta x) \approx (1 - \mu)a + \mu^2 c, \quad (12)$$

which is exact when g is quadratic (power flow in rectangular coordinates) and a good model in polar coordinates. Minimising $\|(1 - \mu)a + \mu^2 c\|_2^2$ and setting the derivative to zero gives the cubic

$$2(c^\top c) \mu^3 - 3(a^\top c) \mu^2 + (a^\top a + 2a^\top c) \mu - a^\top a = 0, \quad (13)$$

and we take the real root in $(0, 2]$ with the smallest model mismatch. NR evaluates $g(x + \Delta x)$ anyway, so the multiplier costs one extra mismatch evaluation when $\mu \neq 1$ and no extra LU. Near the solution $c \rightarrow 0$ and $\mu \rightarrow 1$, so quadratic convergence is kept.

Algorithm 2 NR with the optimal multiplier (improvements/multiplier.py)

Input: x , tolerance 10^{-8} , at most 10 iterations

- 1: **while** $\|g(x)\|_\infty \geq 10^{-8}$ **and** fewer than 10 iterations **do**
- 2: $\Delta x \leftarrow -J(x)^{-1}g(x)$ ▷ one LU
- 3: $a \leftarrow g(x)$, $c \leftarrow g(x + \Delta x)$
- 4: $\mu \leftarrow$ best real root in $(0, 2]$ of the cubic, or 1 if none
- 5: $x \leftarrow x + \mu \Delta x$
- 6: **end while**

7.4 Modification 2: a Newton corrector after every step

Predictor–corrector continuation [15] follows every predictor step with a Newton step back onto the path. Here, after any step to t_{k+1} we take one Newton step on $G(x, t_{k+1}) = 0$:

$$x_{k+1} \leftarrow x_{k+1} - G_x(x_{k+1}, t_{k+1})^{-1} G(x_{k+1}, t_{k+1}), \quad (14)$$

at the cost of one extra LU per path point. On the first step this is again a shifted-Jacobian step; at $t = 1$ it is, exactly, one NR step on g , a fact that matters for how its results are read (Section 10.7).

7.5 Modification 3: Richardson error control of the path steps

The paper chooses its path points by hand; our proposal asked for steps chosen by an error estimate. After the paper's BE step to $t_1 = \Delta t_0$, each attempt from t with step Δt computes one step of size Δt , x_{full} , and two steps of size $\Delta t/2$, x_{half} . For a rule of order p their difference estimates the error of the half steps,

$$\varepsilon = \frac{\|x_{\text{half}} - x_{\text{full}}\|_\infty}{2^p - 1}, \quad x_{\text{new}} = x_{\text{half}} + \frac{x_{\text{half}} - x_{\text{full}}}{2^p - 1}, \quad (15)$$

$$\Delta t \leftarrow \Delta t \cdot \min\left(4, \max(0.1, 0.9(\text{tol}/\varepsilon)^{1/(p+1)})\right). \quad (16)$$

The step is kept if $\varepsilon \leq \text{tol} = 0.5$ (radians and per unit), and then the kept point is the Richardson extrapolation x_{new} ; either way the next Δt is scaled as shown. A step that breaks down (singular matrix or non-finite mismatch) is retried at a quarter of its size. Every attempt costs three steps, so with BE three LUs, and a path that needs more than 150 LUs is stopped and counted as a failure. The step rule is a switch: BE ($p = 1$, the paper's rule), FE ($p = 1$), RK2 ($p = 2$)

or RK4 ($p = 4$), always after a BE first step. The large tolerance is deliberate: the path only has to end inside NR's region of convergence, not on the exact homotopy curve.

Algorithm 3 Richardson-controlled path (improvements/path.py)

```

1:  $x \leftarrow \text{BE}(x^{(0)}, 0, \Delta t_0)$ ;  $t \leftarrow \Delta t_0$ ;  $\Delta t \leftarrow 1 - \Delta t_0$ 
2: while  $t < 1$  and at most 150 LUs used do
3:    $\Delta t \leftarrow \min(\Delta t, 1 - t)$ ;  $x_f \leftarrow \text{STEP}(x, t, \Delta t)$ ;  $x_h \leftarrow \text{STEP}(\text{STEP}(x, t, \frac{\Delta t}{2}), t + \frac{\Delta t}{2}, \frac{\Delta t}{2})$ 
4:   if a step broke down then  $\Delta t \leftarrow \Delta t/4$  and continue
5:    $\varepsilon \leftarrow \|x_h - x_f\|_\infty / (2^p - 1)$ 
6:   if  $\varepsilon \leq \text{tol}$  then
7:      $x \leftarrow x_h + (x_h - x_f) / (2^p - 1)$ ;  $t \leftarrow t + \Delta t$ 
8:   end if
9:    $\Delta t \leftarrow \Delta t \cdot \min(4, \max(0.1, 0.9(\text{tol}/\varepsilon)^{1/(p+1)}))$ 
10: end while

```

7.6 Modification 4: choosing the time points by the mismatch

A cheaper rule controls the step by the quantity NR cares about. It keeps the paper's first step to $t_1 = \Delta t_0$, then tries to jump straight to $t = 1$; a step is kept if it reduced $\|g\|_\infty$, otherwise it is halved (at most six times); after an accepted step the next one is doubled. Rejected trials still count as LUs. Section 10.8 shows the price of using $\|g\|_\infty$ as the test.

Algorithm 4 Adaptive path (improvements/path.py)

```

1:  $x \leftarrow \text{BE}(x^{(0)}, 0, \Delta t_0)$ ;  $t \leftarrow \Delta t_0$ ;  $\Delta t \leftarrow 1 - \Delta t_0$ 
2: while  $t < 1$  do
3:   for up to 7 trials do
4:      $\Delta t \leftarrow \min(\Delta t, 1 - t)$ ;  $\tilde{x} \leftarrow \text{STEP}(x, t, \Delta t)$  ▷ one LU per trial
5:     if  $\|g(\tilde{x})\|_\infty < \|g(x)\|_\infty$  then break
6:     else  $\Delta t \leftarrow \Delta t/2$ 
7:     end if
8:   end for
9:   if no trial was accepted then return failure
10:   $x \leftarrow \tilde{x}$ ;  $t \leftarrow t + \Delta t$ ;  $\Delta t \leftarrow 2\Delta t$ 
11: end while

```

7.7 Modifications 5–8: four ideas that did not work

5. BE-chord. The paper remarks after its eq. (19) that the single fixed-point iteration of BE could be refined. We do three iterations $\hat{x} \leftarrow x_k - \Delta t G_x(x_k, t_{k+1})^{-1} G_t(\hat{x})$, reusing one LU.

6. RK4 with fixed steps. Classical fourth-order Runge–Kutta for the steps after t_1 (the paper's future work; also in our proposal). Under Richardson control it is modification 3 with $p = 4$.

7. Scaled homotopy. $g_0(x) = KD(x - x^{(0)})$ with $D = |\text{diag } J(x^{(0)})|$ normalised to mean 1, so each equation is shifted in proportion to its own diagonal, in the spirit of [4].

8. Newton homotopy. The textbook $G(x, t) = g(x) - (1 - t)g(x^{(0)})$ [14], for which $G_x = J(x)$ and there is no KI term at all.

7.8 Evaluation protocol

The paper tunes its parameters on nine systems. To see whether a modification helps in general, we ran every configuration on all 34 systems under five settings of $(\Delta t_0, K)$, all with the path

$\{0, \Delta t_0, 1\}$ (Figure 19). The multiplier, corrector and adaptive switches were run in all eight on/off combinations; Richardson control with each of BE, FE, RK2 and RK4 and together with the multiplier; and the four other ideas alone. That gives 17 configurations and 2,890 runs. Each configuration is run once, so run times are not compared; cost is the number of LU factorizations.

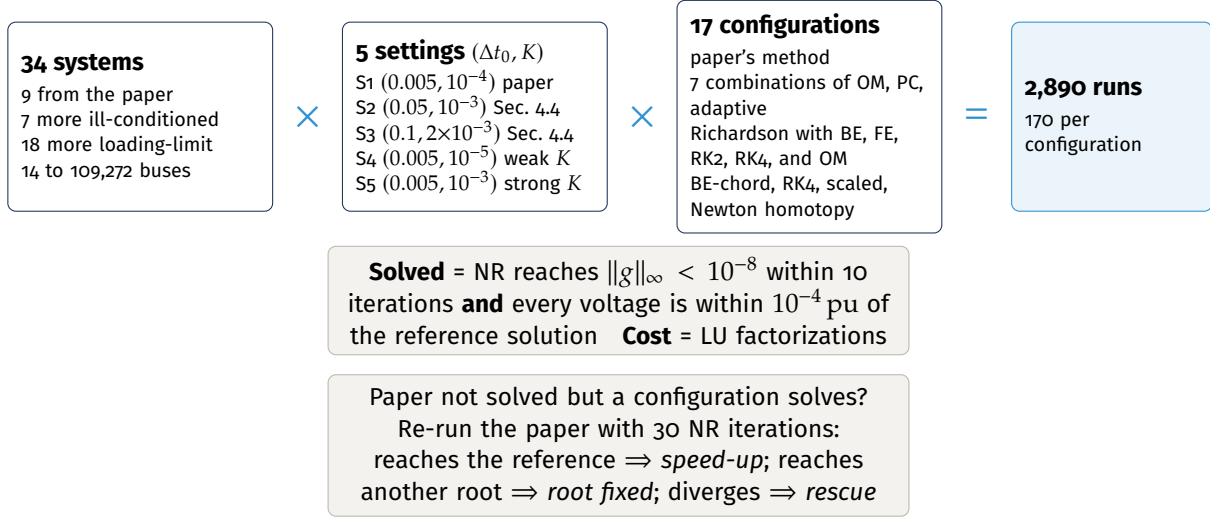


Figure 19. Evaluation protocol for the modifications.

8 Results of the Modifications

8.1 First, which root? The paper's method on the wider test bed

On the 170 runs of the test bed, the paper's method gives NR a start from which it converges within ten iterations 141 times. But 14 of those 141 runs converge to a *different* solution of the power flow equations than the reference operating point of Section 3: case54636 and case109272 with the larger first step of S3 and with the weak or strong K of S4 and S5, case27318 in S4 and S5, case3012wplimit and case3375wplimit already in the paper's own setting S1, and case13659pegaselimit in S4. The mismatch is below 10^{-8} in every one of them, so a convergence test alone cannot tell them apart; only comparing the voltages can. With the paper's own settings all nine of the paper's systems reach the reference, so its published tables are not affected. What is affected is how well the method transfers: it truly solves 127 of 170 runs, not 141. Every number below counts a run as solved only if it reaches the reference, and counts the other-root runs separately.

8.2 Headline numbers

Table 8 and Figure 20 give the result of all 2,890 runs. Every configuration that contains the corrector solves 152 and loses nothing; the multiplier alone solves 143; Richardson control solves 145–158 depending on the step rule, at a much higher cost for the explicit rules. The four other ideas lose runs, fixed-step RK4 catastrophically.

The corrector is the robust improvement, the multiplier the cheap one. The Newton corrector alone solves 25 more runs than the paper's method (2 rescues, 13 wrong roots corrected, 10 speed-ups), never loses one, and cuts the other-root runs from 14 to 2, for 0.4 more LUs per system in S1 (8.43 against 8.04). It also makes the outcome almost independent of Δt_0 : 32

Table 8. Runs that reach the reference solution (of 34 per setting, 170 in total). *Other*: runs that converge to a different root. *Gained* and *lost* are relative to the paper’s method, with gained split into (rescues / root fixed / speed-ups, Section 8.6). *LU_s S1*: mean factorizations in setting S1 on the systems both this configuration and the paper’s method solve (the paper’s own mean on those systems in grey where it is not 8.04). *Ratio*: the same, all five settings pooled, relative to the paper’s method.

Configuration	S1	S2	S3	S4	S5	Total	Other	Gained	Lost	LU _s S1	Ratio
paper	28	27	25	18	29	127	14	–	–	8.04	1.00
PC	32	32	32	22	34	152	2	+25 (2/13/10)	–0	8.43	1.04
OM + PC	32	32	32	22	34	152	3	+25 (2/13/10)	–0	7.57	0.93
OM + PC + adaptive	32	32	32	22	34	152	2	+25 (2/13/10)	–0	7.86	1.25
PC + adaptive	32	32	32	22	34	152	1	+25 (2/13/10)	–0	8.71	1.35
OM	30	30	29	20	34	143	12	+16 (1/5/10)	–0	7.14	0.88
OM + adaptive	32	33	30	20	33	148	10	+22 (4/8/10)	–1	7.46	0.92
adaptive	30	30	26	18	28	132	14	+6 (3/3/0)	–1	8.36	1.03
richardson (BE)	32	34	28	23	34	151	9	+25 (9/7/9)	–1	10.61	1.30
OM + richardson	32	33	34	23	34	156	8	+29 (10/9/10)	–0	9.75	1.18
richardson + FE	34	33	29	27	34	157	3	+31 (9/13/9)	–1	37.14	3.82
richardson + RK2	34	33	30	27	34	158	4	+32 (9/13/10)	–1	47.50	4.78
richardson + RK4	32	30	27	24	32	145	10	+27 (4/13/10)	–9	87.15 (7.96)	8.14
scaled homotopy	28	27	24	18	32	129	6	+3 (0/3/0)	–1	8.04	1.00
BE-chord	19	18	17	14	21	89	24	+9 (0/9/0)	–47	8.89 (8.17)	1.09
newton homotopy	15	15	15	15	15	75	10	+4 (0/4/0)	–56	9.00 (8.13)	1.10
RK4 (fixed steps)	2	3	4	2	2	13	1	+0 (0/0/0)	–114	7.00 (3.00)	2.36

Every configuration on the 34-case test bed (+gained / -lost vs. paper)

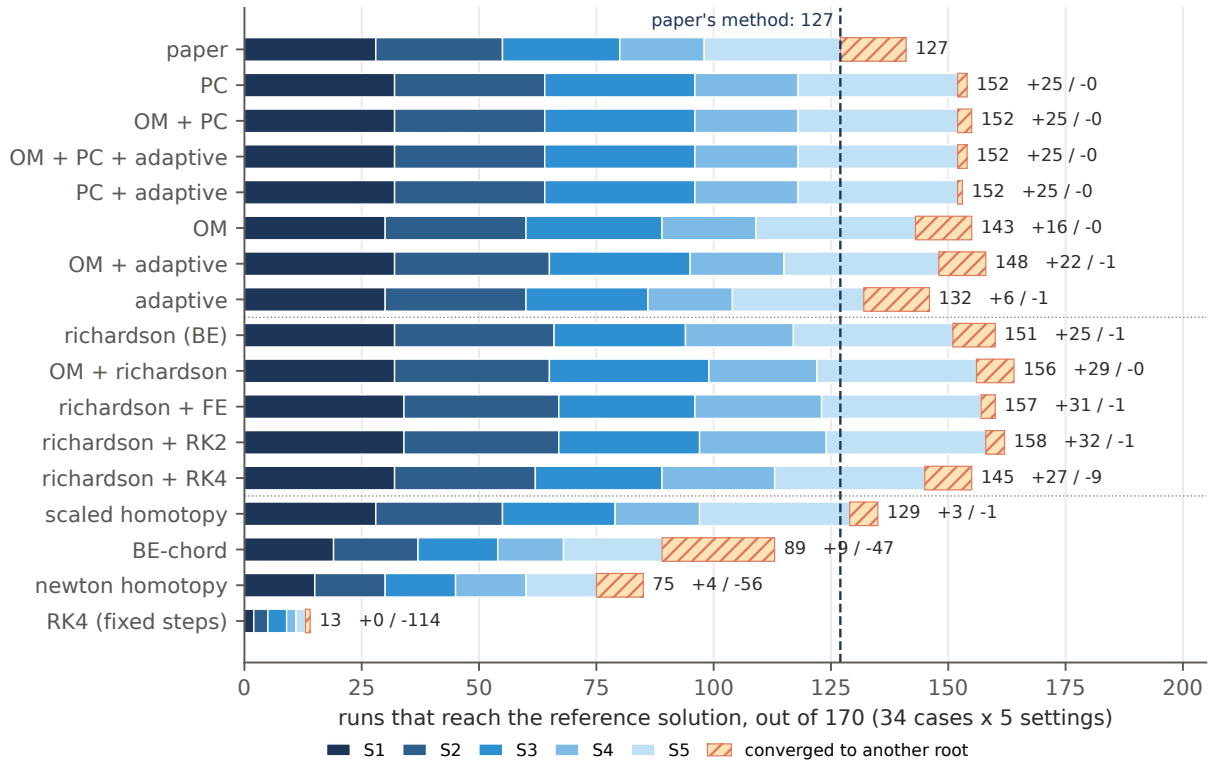


Figure 20. Runs that reach the reference solution per configuration, stacked by setting; the hatched segment is the runs that converge to another root. The dashed line is the paper’s method. Dotted lines separate the corrector/multiplier/adaptive family, Richardson control and the four ideas that did not work.

of 34 systems in each of S1, S2 and S3. The optimal multiplier alone is 11% cheaper (7.14 LUs) but still lands on another root 12 times. Together, **multiplier plus corrector** keeps all of the corrector’s robustness (152 solved, no losses, 3 other-root runs) and most of the multiplier’s saving (7.57 LUs in S1, 7% fewer than the paper’s method over all settings). It is the configuration we recommend.

8.3 Robustness against cost

Figure 21 plots each configuration’s solved count against its relative LU cost. Multiplier plus corrector sits alone in the upper left: more robust *and* cheaper than the paper’s method. The corrector alone is as robust but costs about 5% more, because it adds one LU per path point. Combining the corrector with adaptive steps is expensive (ratio 1.25–1.35): every rejected trial now costs two LUs, and in setting S5 this averages about 20 LUs per run. Richardson control lies to the right: 18–30% more LUs with BE, and 4–8 times the paper’s cost with FE, RK2 and RK4.

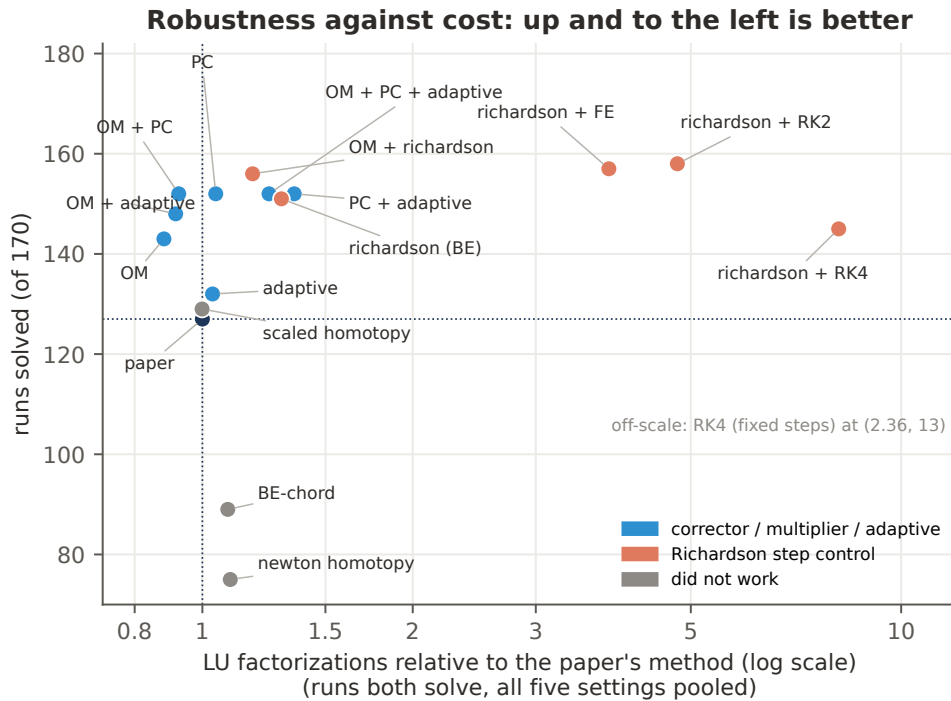


Figure 21. Runs solved against LU cost relative to the paper’s method (log scale). Blue: the corrector, multiplier and adaptive switches and their combinations; orange: Richardson step control with each step rule; grey: the four ideas that did not work.

8.4 Richardson step control

Error-controlled steps are the only change that solves some runs nothing else solves. case6024 in the paper’s own setting S1 is solved only by Richardson control (73 LUs with BE, with or without the multiplier; 106 with FE, 112 with RK2), and case7092, case10595 and case12110 with the weak K of S4, where the paper’s method diverges, are solved only by Richardson configurations. Multiplier plus Richardson rescues 10 of the 11 divergent runs of the test bed—more than any other configuration—and solves 156 runs without losing one.

Figure 22 shows what the controller does on case6024. The paper’s jump from t_1 to $t = 1$ lands at a mismatch NR cannot recover from. The error estimate forces small steps over the early part of the path, where the state changes fastest, and then lets them grow by up to four times per step; the path ends much closer to the solution. The price is visible too: every attempt costs

three steps, and rejected attempts are paid for.

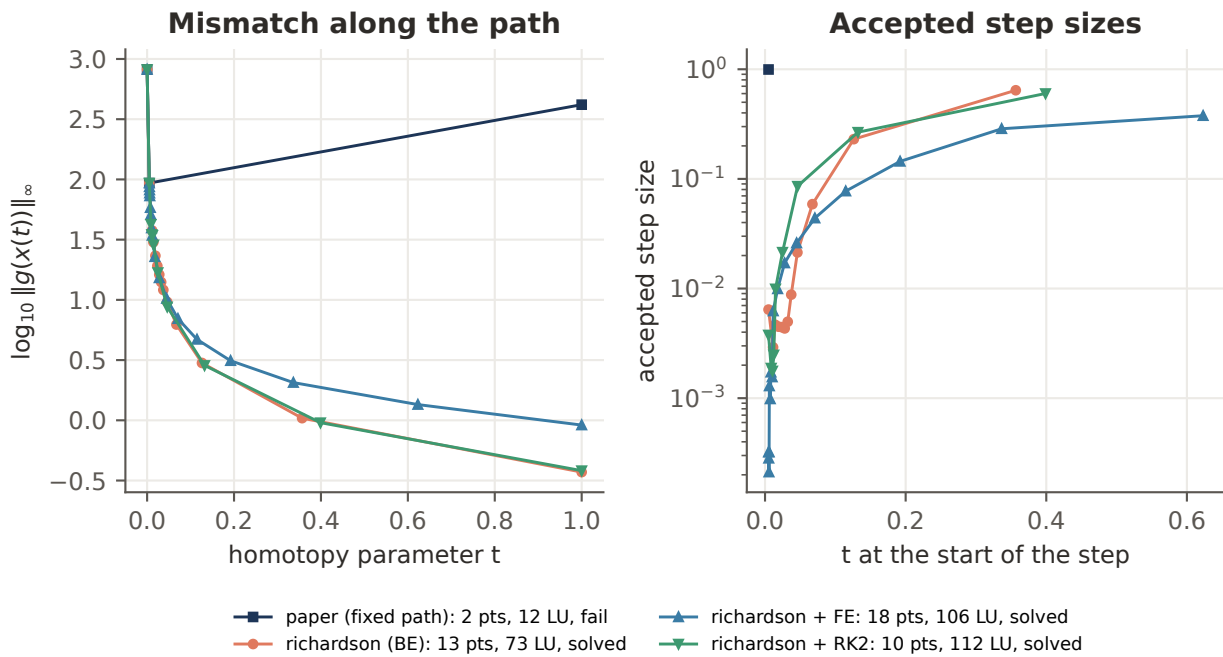


Figure 22. Richardson-controlled paths on case6024, setting S1, the one system in S1 that only Richardson control solves. Left: mismatch at each accepted point (the legend gives the points, the LUs including rejected attempts, and the outcome after NR). Right: the accepted step sizes.

The same study tests the paper’s claim that BE is the right integrator, this time under error control rather than with hand-picked steps. It holds: in S1, Richardson control needs 10.6 LUs with BE, 37 with FE, 48 with RK2 and 87 with RK4. FE and RK2 solve slightly more runs (157 and 158, against 151 for BE)—the extra runs are almost all corrected wrong roots—but at four to six times the cost; RK4 solves fewer (145) and loses nine runs the paper’s method solves. Higher order does not pay here because the steps are limited by stability, not by accuracy (Section 10). Richardson with BE still lands on another root 9 times; the corrector is much better at that (2).

8.5 System by system

Figure 23 shows the LU count of every configuration on every system in the paper’s own setting S1. On the paper’s nine systems nothing much changes: the method already works there with 3–6 LUs, the multiplier saves one LU on case18482 and the corrector adds one on the limit cases. The differences are on the added systems. Of the 14 extra limit cases that both the multiplier and the paper’s method solve, the multiplier saves one to three LUs on 13. On case3012wplimit and case3375wplimit the paper’s method and the multiplier both converge to another root; every configuration with the corrector reaches the reference, with 9–11 LUs. Among the added ill-conditioned systems, case6748 is solved only by adaptive steps (18–19 LUs) and by Richardson control (49 LUs or more), and case6024 only by Richardson control.

8.6 Rescues, root fixes and speed-ups

Because many limit cases sit exactly on NR’s 10-iteration budget (Section 3), a configuration can “solve” a system just by saving one iteration. We therefore re-ran the paper’s method with 30 NR iterations on every run it does not solve that some configuration does. There are 36 such runs, and they are of three kinds:

- **10 speed-ups.** case14limit and case2736splimit in every setting: the paper’s method

	paper	PC	OM + PC	OM + PC + adaptive	PC + adaptive	OM	OM + adaptive	adaptive	richardson (BE)	OM + richardson	richardson	richardson + FE	richardson + RK2	scaled homotopy	BE-chord	newton homotopy	RK4 (fixed steps)	
case18482	6	6	6	6	6	5	5	6	7	7	43	46	88	6	o	x	x	paper's 9 cases
case27318	6	6	6	6	6	6	6	6	7	7	29	46	88	6	o	8	x	
case36964	6	6	6	6	6	6	6	6	13	13	46	46	88	6	x	x	x	
case54636	6	6	6	6	6	6	6	6	7	7	29	46	88	6	o	8	x	
case109272	6	6	6	6	6	6	6	6	7	7	29	46	88	6	o	o	x	
case69limit	3	4	4	4	4	3	3	3	5	5	8	9	15	3	3	3	7	
case141limit	3	4	4	4	4	3	3	3	5	5	8	9	15	3	3	3	7	
case500limit	4	5	5	5	5	4	4	4	6	6	28	34	63	4	5	5	x	
case2000limit	5	6	6	6	6	5	14	14	6	6	34	51	87	5	6	6	x	
case6024	x	x	x	x	x	x	x	x	73	73	106	112	x	x	x	x	x	7 extra ill-conditioned
case6243	6	6	6	6	6	6	6	6	7	7	43	46	88	6	x	x	x	
case6748	x	x	x	x	x	x	18	19	49	49	52	52	100	x	x	x	x	
case7092	6	6	6	6	6	6	6	6	7	7	40	46	76	6	7	x	x	
case9961	6	6	6	6	6	6	6	6	7	7	40	46	88	6	7	x	x	
case10595	6	6	6	6	6	5	5	6	7	7	46	46	88	6	o	x	x	
case12110	5	6	6	6	6	5	5	5	7	7	46	52	88	6	6	x	x	
case14limit	x	13	10	10	13	10	10	x	14	12	34	57	129	x	x	x	x	18 extra limit cases
case_ieee30limit	10	11	9	9	11	9	9	10	11	10	32	61	102	10	11	11	x	
case57limit	11	11	9	9	11	9	9	11	12	10	31	53	139	10	11	11	x	
case89pegaselimit	11	11	9	9	11	9	9	11	12	10	32	54	141	11	12	12	x	
case118limit	10	11	9	9	11	9	9	10	11	9	43	54	o	10	11	11	x	
case300limit	10	11	9	9	11	8	8	10	11	9	37	49	78	10	11	11	x	
case1354pegaselimit	12	12	10	10	12	9	9	12	13	10	44	55	104	12	12	12	x	
case2383wplimit	10	11	10	18	19	9	9	10	11	10	40	54	78	10	11	11	x	
case2736splimit	x	13	9	9	13	11	11	x	14	12	45	51	105	x	x	x	x	
case2737soplmit	12	12	10	10	12	10	10	12	13	11	47	62	92	12	x	x	x	
case2746woplmit	12	12	10	10	12	10	10	12	13	11	47	50	92	12	12	x	x	
case2746wplimit	12	12	10	10	12	10	10	12	13	11	44	56	116	12	x	x	x	
case2869pegaselimit	10	11	10	10	11	9	9	10	11	10	42	54	90	10	11	11	x	
case3012wplimit	o	11	10	10	11	o	18	19	o	o	53	56	116	o	x	x	x	
case3120splimit	12	12	10	10	12	10	10	12	13	10	47	56	92	12	x	x	x	
case3375wplimit	o	11	9	9	11	o	o	o	o	o	53	50	91	o	12	x	x	
case9241pegaselimit	11	11	10	10	11	9	9	11	46	45	52	55	91	11	11	12	x	
case13659pegaselimit	8	9	8	8	9	8	8	8	9	9	33	48	90	8	10	o	x	

Figure 23. LU factorizations per system and configuration in setting S1 (the paper's parameters). **x**: fails; **o**: converges to another root. Bold numbers are cheaper than the paper's method or solve a system it does not. White lines separate the paper's systems, the extra ill-conditioned systems and the extra limit cases, and the configuration families.

reaches the reference in 11 NR iterations, one more than allowed.

- **15 root fixes.** The paper's method converges, but to another root (Section 8.1). The corrector fixes 13 of them; the other two (case109272 and case13659pegaselimit in S4) only Richardson control with an explicit step rule fixes.
- **11 rescues.** The paper's method diverges even with 30 iterations: case36964 in S2 and S3, case6024 in S1–S3, case6748 in S1–S3, and case7092, case10595 and case12110 in S4. Multiplier plus Richardson rescues 10 of them, Richardson with BE 9, multiplier plus adaptive 4, the corrector 2 and the multiplier 1.

The remaining 7 runs the paper's method does not solve are solved by nothing (Section 10.11).

Figure 24 shows one run of each kind in full. On case36964 in S2 the paper's NR wanders and diverges; the multiplier converges in six iterations and the corrector in two. On case3012wplimit the paper's method and the multiplier converge, in 10 and 9 NR iterations, but to the wrong root. The corrector reaches the reference in 7; our reading is that its extra Newton step at t_1 keeps the path on the branch that leads there. On case14limit the paper's method converges one iteration too slowly; every switch brings it within the budget.

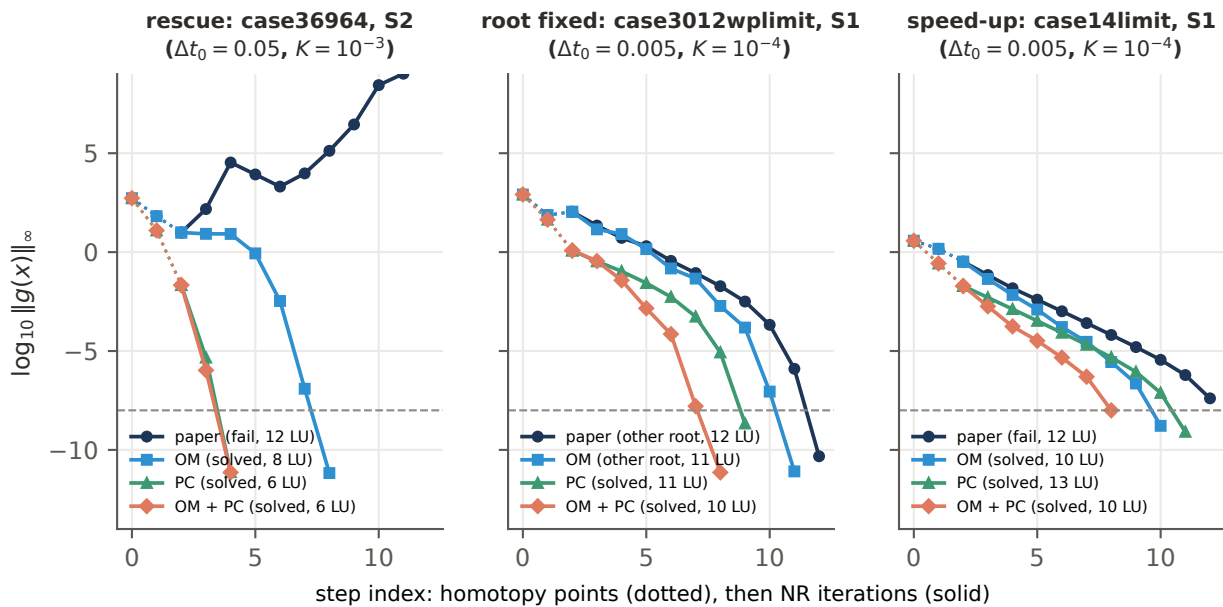


Figure 24. Mismatch along the path (dotted) and through NR (solid) for three runs the paper's method does not solve, one of each kind. The legend gives the outcome (solved means the reference solution) and the LU count. Dashed line: tolerance 10^{-8} .

Figure 25 shows how the adaptive rule handles case6748. The direct jump from t_1 to $t = 1$ lands exactly where the paper's fixed path does, at a mismatch of 1798, and is rejected; the rule halves the step six times, finds that only a step of 0.0155 reduces the mismatch, and from there doubles its way to $t = 1$ in eight points. NR then needs three iterations. The whole solve costs 18 LUs—three times the paper's method on an easy system, but the paper's method does not solve this one at all.

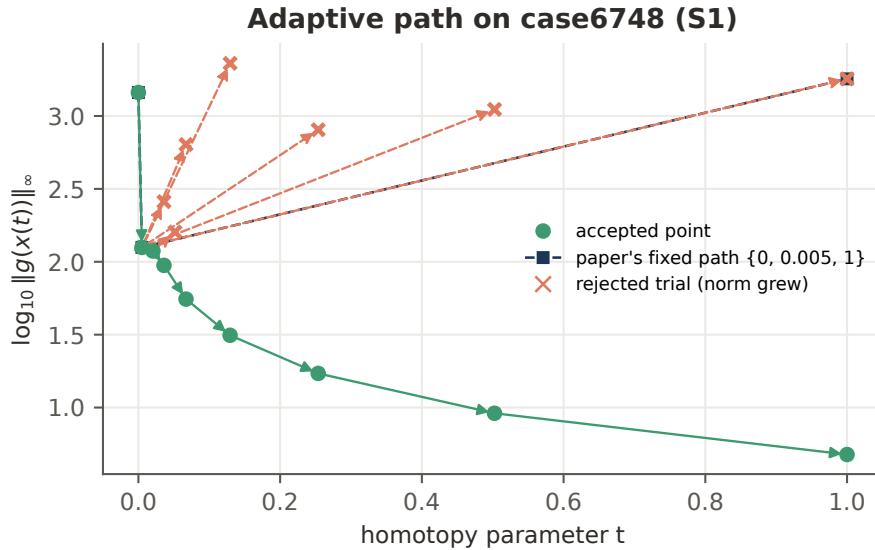


Figure 25. Every step the adaptive rule (with the multiplier) tries on case6748, setting S1. Green: accepted steps; orange: rejected trials, which still cost an LU each. The navy square path is the paper's fixed path, which ends at the same point as the first rejected jump.

9 The Proposal's Numerical Studies

Our proposal promised four numerical studies around the paper's central parameter, the shift $\delta = K/\Delta t_0$ of the first step. They live in the extensions package, each has its own experiment (exp12–exp15), and each can be run on any system from the command line:

Listing 3 The proposal studies from the command line

```
python -m extensions spectrum case18482 # power method and inverse iteration
python -m extensions feasibility case6024 # (K, dt0) map as text
python -m extensions tune case6024 # golden-section search for delta and dt0
python -m extensions scratch case_ACTIVSG500limit --refiner FDXB # our own LU
```

9.1 The spectrum of the flat-start Jacobian

The first BE step solves with $J + \delta I$ (Equation (10)), whose eigenvalues are those of J moved by δ . How large δ must be depends on how close the eigenvalues of $J(x^{(0)})$ come to zero.

Method. The largest eigenvalue modulus comes from the power method. J is not symmetric and its dominant eigenvalues can come as a complex pair or a $\pm\lambda$ pair, for which the plain iteration oscillates, so we iterate with J applied twice and take $\sqrt{\|J^2 x\|/\|x\|}$. The smallest modulus comes from inverse iteration: the same power method applied to $(J - \sigma I)^{-1}$, using one sparse LU of $J - \sigma I$ and then only triangular solves. With $\sigma = 0$ it gives $|\lambda|_{\min}(J)$, and with $\sigma = -\delta$ the smallest eigenvalue modulus of $J + \delta I$. ARPACK (`scipy.sparse.linalg.eigs`) is used as an independent check. The ratio $|\lambda|_{\max}/|\lambda|_{\min}$ is a lower bound on the 2-norm condition number of J .

What it shows. On the twelve ill-conditioned systems $|\lambda|_{\min}(J)$ is 4×10^{-4} to 6×10^{-3} , while $|\lambda|_{\max}$ is 2.7×10^4 to 5.2×10^4 , so the eigenvalue ratio reaches 1.2×10^8 (case109272). The paper's $\delta = 0.02$ lifts the smallest eigenvalue to about 0.02 and cuts the ratio to 1.1 – 2.3×10^6 on all twelve, a factor of 4 to 50 (Figure 26). On most limit cases the smallest eigenvalue is already 0.01 to 1.9, as large as the shift or larger, so the shift hardly changes anything—which is why the paper's method

behaves like plain NR on those systems. The two exceptions are the limit cases built from the two largest PEGASE grids, `case9241pegaselimit` and `case13659pegaselimit`, whose spectrum looks like that of the ill-conditioned replicas.

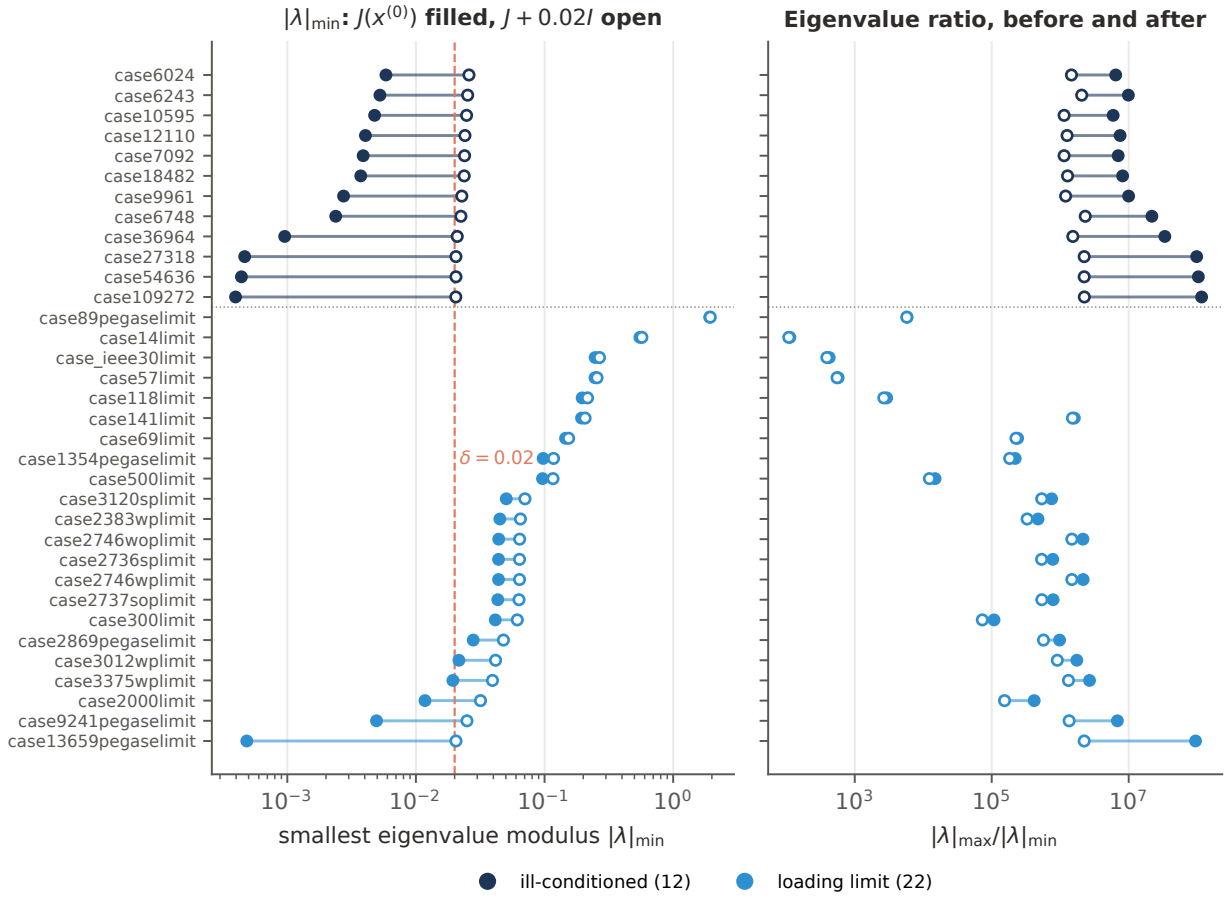


Figure 26. Smallest eigenvalue modulus (left) and the ratio $|\lambda|_{\max}/|\lambda|_{\min}$ (right) of $J(x^{(0)})$ (filled) and of the shifted matrix $J + 0.02I$ of the first BE step (open), for all 34 systems. The shift only matters where $|\lambda|_{\min}$ is below δ : on the ill-conditioned systems and the two large PEGASE limit cases.

How accurate the iterations are. Inverse iteration agrees with ARPACK to four digits on 33 of the 34 systems (on `case571limit` it is 8% off). The power method is less reliable: it is off by up to 19% (`case10595`), and all its large errors are on the networks built from copies of the same grid. There every copy contributes an eigenvalue of almost the same modulus, so the ratio of the two largest moduli is close to one and the power method, whose error falls like that ratio to the power of the iteration count, has not converged after its 500 iterations. The textbook convergence condition of the power method is violated by construction of the test systems.

9.2 Where the method works in the $(K, \Delta t_0)$ plane

The paper tries three $(\Delta t_0, K)$ pairs. Figure 27 runs its method (BE on $\{0, \Delta t_0, 1\}$, then NR) on an 8×11 grid, Δt_0 from 0.001 to 0.2 and K from 10^{-6} to 10^{-1} , for four systems, and marks whether each run reaches the reference solution, another root, or fails. Lines of constant δ are diagonals of these maps.

The method works in a band along the diagonals $\delta = \text{const}$, which confirms that δ and not K or Δt_0 alone is the parameter that matters (Section 4.3). How wide the band is depends on the system. For `case18482` and `case2000limit` almost the whole grid works. For `case36964` only 29 of 88 cells work, and no cell with $\Delta t_0 = 0.1$ or 0.2 works: this is why the paper found that

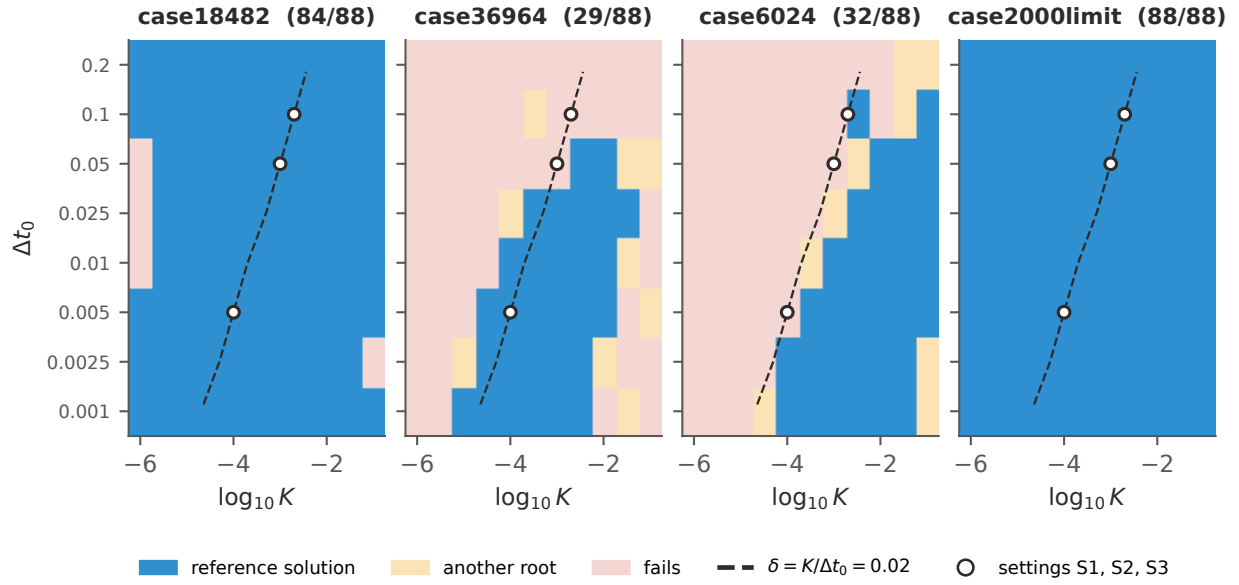


Figure 27. Outcome of the paper's method on a grid of K and Δt_0 (the title gives the cells that reach the reference solution). Dashed: $\delta = 0.02$, the paper's shift. Circles: the settings S1, S2 and S3 of the modification study.

$\Delta t_0 = 0.1$ with $K = 0.002$ fails even though δ is unchanged (Section 6). For case6024 the band lies at larger shifts, δ from 0.03 to 32, and the paper's $\delta = 0.02$ line runs along its edge, where runs reach other roots; the paper's own setting fails there. Both of the systems with a narrow band have nine cells that converge to another root, mostly at the edges of the band.

9.3 Tuning δ by golden-section search

Method. The objective is the total LU count of the paper's method, with a failed run costing 50, plus a small tie-break $0.1 \log_{10} \|g(x(1))\|_\infty$ so that the search can move inside a plateau of equal integer counts. Golden-section search runs first over $\log_{10} \delta \in [-4, 1]$ at $\Delta t_0 = 0.005$, then over $\log_{10} \Delta t_0 \in [-3, -0.5]$ with that δ , each to a bracket of 0.05 decades, for 21 solves per system in total. The objective is piecewise constant and not unimodal, so the search returns the best point it has seen rather than the last bracket.

What it shows. The tuned values tell a different story from the paper's single δ . For case27318, case54636 and case109272 the search lands at $\delta = 0.025$, close to the paper's 0.02; the paper's choice suits its own systems. On the other ill-conditioned systems it prefers $\delta = 1$ to 5, fifty times larger, and on most limit cases it runs to the lower end of the interval, because there δ hardly matters (Figure 28). At the tuned point 32 of the 34 systems are solved; the two that are not are case141limit and case2736splimit, which need 11 NR iterations whatever the start.

We then fitted two rules on the paper's nine systems and tested them on the other 25 (Table 9): a fixed δ , the geometric mean of the tuned values, and a *spectral rule* $\delta = c |\lambda|_{\min}$, with c the geometric mean of $\delta/|\lambda|_{\min}$, which costs one extra LU for the inverse iteration. The fixed rule gives $\delta = 0.011$ and behaves like the paper's 0.02. The spectral rule does not work: on the ill-conditioned systems the tuned δ is 54 to 819 times $|\lambda|_{\min}$, so no single factor fits, and on the limit cases δ barely matters. It loses four of the nine training systems. Searching each system separately solves four more test systems than the paper's δ , but the search costs 21 solves, far more than it saves.

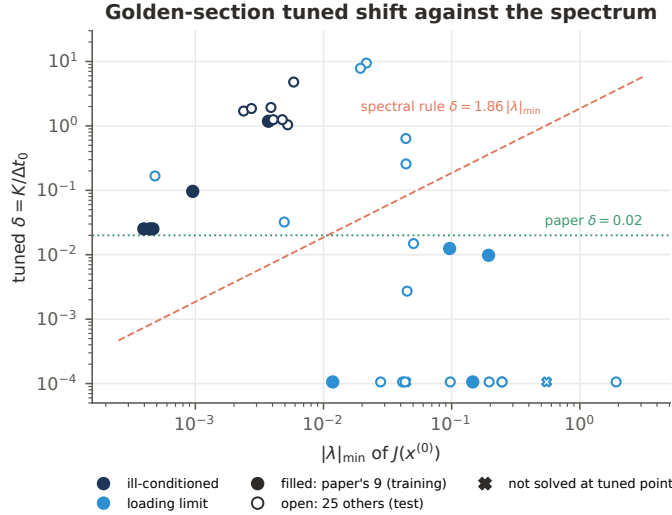


Figure 28. Golden-section tuned δ against $|\lambda|_{\min}(J(x^{(0)}))$ for all 34 systems. Points at 10^{-4} are systems where the search ran to the lower end because δ does not matter. A spectral rule would put the points on a line of slope one; they are nowhere near one.

Table 9. Choosing δ without a search: rules fitted on the paper's nine systems, tested on the other 25 ($\Delta t_0 = 0.005$ except for the per-system search). Mean LUs are over the solved systems and exclude the cost of the search itself.

How δ is chosen	Solved, training	LUs	Solved, test	LUs
paper, $\delta = 0.02$	9 of 9	5.00	19 of 25	9.47
fixed rule, $\delta = 0.011$	9 of 9	5.11	19 of 25	9.53
spectral rule, $\delta = 1.86 \lambda _{\min}$	5 of 9	5.20	20 of 25	11.00
golden search per system (21 solves)	9 of 9	4.89	23 of 25	9.13

9.4 Linear solvers written from scratch

Method. `extensions/gauss.py` implements Gauss elimination with partial pivoting and back substitution, and a Doolittle LU factorization with partial pivoting, $PA = LU$, with L (unit diagonal) and U stored in one array and the row permutation as an index vector, followed by forward and back substitution. The row updates are vectorised, but both methods work on dense arrays and cost $O(n^3)$. `extensions/scratch.py` repeats the paper's BE path, NR and FDXB with every linear system solved this way, step for step as in `dynhomotopy`, so the two can be compared one to one.

What it shows. On the tutorial and on the limit cases of 69 to 2,000 buses, our LU gives exactly the same iteration counts as SuperLU and the same solution to 5×10^{-12} (Table 10). The time shows why the paper's systems need a sparse solver: NR on `case2000limit` takes 424s with our LU against 0.04s with SuperLU. From 943 to 3,607 unknowns the time grows 123-fold, more than the $3.8^3 = 56$ of pure $O(n^3)$ scaling, presumably because the larger matrices no longer fit in cache. The dense Jacobian of `case109272` alone would need $185,807^2 \times 8$ bytes, about 276 GB. Reuse of the factors matters as well: for 20 right-hand sides with one 400×400 matrix, one LU plus 20 pairs of triangular solves takes 0.054s against 0.747s for 20 Gauss eliminations, 14 times less, which is why FDXB factorises B' and B'' once.

What the numerical studies establish. The shift δ matters exactly where the spectrum says it should: on systems whose flat-start Jacobian has eigenvalues below δ . There the method works in a band of constant δ whose position and width depend on the system, and the paper's $\delta = 0.02$ sits well inside the band for some systems and on its edge for others. Neither a fixed value nor a multiple of $|\lambda|_{\min}$ predicts the best δ ; a per-system search does,

Table 10. The paper’s method (BE path $\{0, 0.005, 1\}$, $K = 10^{-4}$) with SciPy’s sparse LU and with our dense LU with partial pivoting: iterations of the refiner, largest difference in the solution, and run time.

System	Refiner	Its SuperLU	Its ours	$\max \Delta x $	SuperLU (s)	Ours (s)
tutorial (Sec. 3.3)	NR	9	9	0	0.002	0.002
case69limit	NR	1	1	3.6×10^{-13}	0.004	0.011
	FDXB	8	8	1.6×10^{-13}	0.004	0.010
case141limit	NR	1	1	9.7×10^{-14}	0.005	0.039
	FDXB	8	8	4.7×10^{-12}	0.004	0.032
case500limit	NR	2	2	9.8×10^{-15}	0.010	3.45
	FDXB	13	13	1.2×10^{-14}	0.008	1.71
case2000limit	NR	3	3	4.3×10^{-14}	0.040	424.1
	FDXB	22	22	6.2×10^{-14}	0.029	100.8

but costs more than it saves. This is why the corrector, which makes more settings work (Section 8) instead of looking for a better one, is worth more than tuning δ .

10 Failure Analysis

This section collects every failure—of the paper’s integrators, of the modifications that did not work, of the proposal ideas that did not deliver, and of our own measurements—and explains each from the equations where we can. Where an explanation is our interpretation rather than something we measured directly, we say so.

10.1 Why explicit schemes fail on the first step

Forward Euler at $t = 0$ inverts $G_x = KI$ and therefore steps by $-(\Delta t_0/K) g(x^{(0)})$ (Section 4.3). The step is proportional to the mismatch and blind to the network, and the ill-conditioned systems start with mismatches of 200–530. RK2 evaluates its second slope at that FE point, so it inherits the damage. This is not a question of step size: making $\Delta t_0/K$ small enough to be safe makes the step useless, which is why the paper and our code always take the first step with BE.

10.2 Why explicit schemes still fail with large later steps

After t_1 , an explicit step to $t = 1$ uses $G_x = t_1 J + (1 - t_1) KI$ evaluated at $t_1 = 0.005$, a matrix that is still dominated by KI in every direction where J is small, so the same blindness returns in those directions. BE instead uses G_x at $t = 1$, that is J itself (Section 4.3). The ODE is stiff in exactly the sense of [16]: its fast modes, with time scale $K/\lambda(J)$, are far faster than the path, and only an implicit method can step over them. More path points help FE and RK2 (Table 5, Figure 11) because they reduce the step at which the stale G_x is used. The spectrum makes the stiffness concrete: with $|\lambda|$ from 4×10^{-4} to 5×10^4 (Figure 26), the time scales $K/|\lambda|$ of the modes span eight orders of magnitude.

10.3 Why RK4 fails, and why higher order does not pay under error control

With fixed steps RK4 solves 13 of 170 runs. It is explicit, and its intermediate stages sit at $t + \Delta t/2$ and $t + \Delta t$ using slopes computed from explicit predictions; with $\Delta t \approx 1$ each stage has the FE failure mode above, and higher order only multiplies the number of bad evaluations (four LUs

per step). Order of accuracy is irrelevant when the step is outside the stability region.

Richardson control repairs this by shrinking the step until the estimate is small, and then RK4 does solve 145 runs—but the step it can take is set by stability, not by accuracy, so it is no larger than FE's, and each attempt now costs twelve LUs (three steps of four stages). That is why it needs 87 LUs in S1 against 37 for FE and 10.6 for BE, and why the paper's preference for BE survives a fair, error-controlled comparison. Richardson with RK4 is also the only Richardson configuration that loses more than one run the paper's method solves (nine): on six of them it converges to another root, and on three NR fails from where the path ends. More stages do not make the path more faithful when the step is limited by stability.

10.4 Why refining backward Euler makes it worse

The paper suggests after its eq. (19) that more fixed-point iterations could improve BE. BE-chord loses 47 runs and lands on another root 24 times. The reason is the contraction condition of the iteration it performs,

$$\hat{x} \mapsto x_k - \Delta t G_x(x_k, t_{k+1})^{-1} G_t(\hat{x}), \quad \frac{\partial}{\partial \hat{x}} = -\Delta t G_x(x_k, t_{k+1})^{-1} (J(\hat{x}) - KI).$$

On the last step $t_{k+1} = 1$, so $G_x = J(x_k)$, and when $J(\hat{x}) \approx J(x_k)$ the iteration matrix is close to $-\Delta t I$ with $\Delta t = 0.995$: its spectral radius is about one, so the iteration does not contract—it oscillates. The first iterate is useful (it is the near-Newton step of Section 4.3); the second and third move back and forth around it. For small steps the same iteration would contract, which is presumably the regime the paper had in mind. (This is our analysis of why the iteration drifts; we measured the outcome, not the spectral radius.)

10.5 Why the Newton homotopy fails

The Newton homotopy $G = g(x) - (1 - t)g(x^{(0)})$ has $G_x = J(x)$ at every t : no KI term. Its BE first step is $\Delta x = -\Delta t_0 J(x^{(0)})^{-1} g(x^{(0)})$, a plain Newton step shrunk to 0.5%, with the flat-start Jacobian unregularised. It solves 75 of 170 runs; in setting S1 it fails on every ill-conditioned system except case27318 and case54636, and on case109272 it converges to another root. This is the clearest evidence in the project for the paper's central idea: the useful ingredient is the KI shift of the Jacobian, not homotopy continuation as such.

10.6 Why the scaled homotopy changes little

Scaling the shift by $D = |\text{diag } J(x^{(0)})|$ gives exactly the paper's result in S1, S2 and S4, loses one run in S3 (case27318), and in S5 corrects the wrong roots of case27318, case54636 and case109272: 129 solved in total, against 127. Our interpretation of why it does not help more: D is normalised to mean one, so equations with a small Jacobian diagonal get a *smaller* shift than before—and those weakly coupled equations are exactly the ones the shift is there to protect. A scaling that raises the shift where the diagonal is weak would be the natural next try.

10.7 Why part of the corrector's gain is accounting

At $t = 1$ the homotopy is $G(x, 1) = g(x)$ and $G_x(x, 1) = J(x)$, so the corrector's last step is *exactly* one NR step, taken before the 10-iteration budget starts counting. Ten of the corrector's 25 gains are the budget-edge speed-ups of Section 8.6. On case14limit and case2736splimit the corrector solves with 13 LUs—exactly what the paper's method needs if it is simply allowed its eleventh NR iteration (2 + 11). So on those two systems the corrector is not cheaper and not more robust; it only moves one iteration outside the count. The other 15 gains are real: its two rescues (case36964 in S2 and S3, where at 6 LUs it is the cheapest solver we have) and its 13 root fixes,

which no count of iterations can explain. The LU metric makes the difference visible; a pure success count hides it.

10.8 Where adaptive steps lose

The adaptive rule accepts a step only if $\|g\|_\infty$ decreases. But the homotopy path does not promise that: on the path, $g(x(t)) = -\frac{1-t}{t}K(x(t)-x^{(0)})$, whose norm need not fall monotonically. So the rule can reject perfectly good steps. That is what happens on `case2000limit` in S1: the paper's direct jump works with 5 LUs, the adaptive rule rejects it and spends 14. It also explains the one run adaptive loses, `case2383wplimit` with strong K (S5), where it runs out of halvings. Measuring progress by $\|G(x, t)\|$ instead of $\|g(x)\|$ would respect the path; we did not test it. Richardson control avoids the problem because it measures the integration error, not the mismatch—at a higher price per step.

10.9 Why Richardson control still finds wrong roots

Richardson with BE lands on another root 9 times, the corrector twice. The error estimate compares two integrations with each other; it does not measure how far either is from the homotopy curve, and with the tolerance of 0.5 that we chose (so that paths stay cheap) both can drift together towards another branch. The corrector measures exactly that distance, $G(x, t)$, and pulls the state back. (This is our explanation; we did not vary the tolerance.) Combining the two would be the obvious next step, but it would add one more LU to each of the three steps of every attempt.

10.10 Why no simple rule predicts δ

The spectral rule assumes that the right shift is proportional to $|\lambda|_{\min}(J)$. The feasibility maps show why it cannot be: the band of working δ lies at 0.006–3 for `case36964` and at 0.03–32 for `case6024`, although their smallest eigenvalues differ by a factor of six in the other direction (10^{-3} against 6×10^{-3}). What decides whether the first step lands in NR's region of convergence depends on the whole nonlinear problem, not on one eigenvalue of the linearisation at the start. The smallest eigenvalue does say something useful, but only as a yes/no test: if it is well above 0.02, δ does not matter.

10.11 The seven runs nothing solves

Seven runs fail under every configuration, all in setting S4, with the weak $K = 10^{-5}$ ($\delta = 0.002$, ten times less regularisation than the paper's): `case36964`, `case6024`, `case6243`, `case6748`, `case9961`, `case3012wplimit` and `case3375wplimit`. S4 is the setting where the paper's method drops to 18 of 34. Richardson control with an explicit step rule gets furthest (27 of 34), but no switch fully compensates for too small a shift.

10.12 Failures of our own measurement

Three measurement problems were ours, not the method's.

- **Counting convergence as success.** Our first version of the modification study counted a run as solved when NR converged. Under that criterion the paper's method solved 141 runs and the multiplier looked like the best single change. Checking the voltages against the reference operating point showed that 14 of the paper's converged runs, and 12 of the multiplier's, reach another root, and it reversed the ranking of the multiplier and the corrector. Every result in this report uses the stricter criterion.

- **Budget-edge systems.** The criterion “NR within 10 iterations” is blunt on systems that need exactly 11, which is why Section 8.6 re-runs every gained case with 30 iterations before calling it a rescue.
- **Wall-clock noise.** Times on a laptop depend on its load; we report medians of seven runs, compare only percentages with the paper, and base all our own cost conclusions on LU counts.

11 Discussion

11.1 Strengths

The reproduction is close and fully traceable: every number in this report comes from a script in the repository that prints our value next to the paper’s, and the tests check the building blocks against independent computations before any comparison. The analysis of Section 4.3, that the paper’s method is a shifted Newton step followed by a near-Newton step, turns an empirical recommendation into an explanation, and it predicted which of our modifications would work: the ones that improve the Newton stages (corrector, multiplier) helped, and the one that removes the shift (Newton homotopy) failed. The modification study is a controlled experiment on 34 systems, 25 of them unseen by the paper, with a machine-independent cost measure; it checks that each run reaches the intended solution, and it separates rescues, root fixes and speed-ups. The proposal studies connect the method to the numerical analysis behind it: the spectrum explains where the shift matters, the feasibility maps show that δ is the right parameter, and our own LU confirms the linear algebra independently of SciPy.

11.2 Limitations

- The test bed is two Zenodo records. The ill-conditioned systems are replicas of a few base networks, so 12 systems carry less independent evidence than 12 unrelated grids would.
- The reference operating point is a convention: the solution NR reaches from the stored initial guess. We did not study whether the other roots are physically meaningful operating points.
- Only paths $\{0, \Delta t_0, 1\}$ were used for the modification study; the modifications were not tested on the multi-point paths of the paper’s Tables 5 and 6.
- The Richardson tolerance (0.5) and the golden-section brackets were set once and not tuned; the feasibility maps cover four systems.
- Our GSH-NR is a reconstruction from a verbal description, so the comparison with it is weaker than the rest.
- Timings are Python on a laptop and are noisy; they are reported but not relied on.

11.3 Future work

- **Corrector plus error control.** Richardson control rescues the most divergent runs and the corrector fixes the most wrong roots; a Richardson path with a corrector after each accepted point would try to get both.
- **Cheaper error estimates.** An embedded pair, or reusing one LU for the three steps of each attempt, would cut the cost of Richardson control by up to a factor of three.

- A **path-aware acceptance test** for adaptive steps, based on $\|G(x, t)\|$ rather than $\|g(x)\|$ (Section 10.8).
- A **scaling that raises the shift on weak equations**, the opposite of our scaled homotopy.
- A **sparse LU from scratch**, with a fill-reducing ordering, to close the gap between our dense solver and SuperLU and run the whole pipeline on our own linear algebra.

12 Conclusion

We set out to rebuild a published method for starting Newton–Raphson on ill-conditioned power grids and to see how far it could be trusted and pushed. It reproduces: on all five ill-conditioned grids of up to 109,272 buses, NR from the flat start fails, and a two-point backward-Euler homotopy turns that into a four-iteration solve, with our numbers matching the paper’s across 28 orders of magnitude. It is also simpler than its derivation suggests. With the paper’s parameters, the first backward-Euler step is a Newton step on a Jacobian shifted by $\delta = K/\Delta t_0$, and the second is almost a plain Newton step; the implicit scheme wins because it is the only one that uses the real Jacobian where it matters.

On a test bed four times the size of the paper’s, one question turned out to matter more than any other: which solution does a converged run reach? The paper’s method converges on 141 of 170 runs but reaches the intended operating point on 127. A Newton corrector after every path step raises that to 152, loses nothing, and cuts the wrong-root runs from 14 to 2; with Iwamoto’s optimal multiplier added it does so with 7% fewer LU factorizations than the paper’s method, and this is the configuration we recommend. Richardson error control of the steps solves systems nothing else solves, at 18–30% more factorizations, and confirms under fair conditions that backward Euler is the right integrator. Four other ideas failed, each for a reason in the equations: RK4 is explicit, extra fixed-point iterations do not contract at a step of one, a scaled shift weakens the equations that need it, and a homotopy without the shift does not work at all.

The proposal’s studies explain the parameter behind all of this. The shift matters where the flat-start Jacobian has eigenvalues below it, the method works in a band of constant δ whose position differs from system to system, and neither a fixed value nor a spectral rule predicts the best δ . Our own Gauss elimination and LU reproduce every iterate, and show at 424 seconds for a 2,000-bus system why a sparse solver is not optional.

The lesson we take from the project is that the choice of what to count decides the conclusions. Counting LU factorizations instead of seconds, asking whether a “solved” run was rescued or merely hurried, and above all checking that it reached the right solution each changed a conclusion that a plain success count would have got wrong.

References

- [1] A. Lima-Silva and F. D. Freitas, "Exploring a dynamic homotopy technique to enhance the convergence of classical power flow iterative solvers in ill-conditioned power system models," *Energies*, vol. 17, no. 18, art. 4642, 2024. doi:10.3390/en17184642.
- [2] A. Lima-Silva and F. D. Freitas, "Dynamical homotopy transient-based technique to improve the convergence of ill-posed power flow problem," *Int. J. Electr. Power Energy Syst.*, vol. 155, art. 109436, 2024.
- [3] F. D. Freitas and A. L. Silva, "Flat start guess homotopy-based power flow method guided by fictitious network compensation control," *Int. J. Electr. Power Energy Syst.*, vol. 142, art. 108311, 2022.
- [4] F. D. Freitas and L. N. de Oliveira, "Conditioning step on the initial estimate when solving ill-conditioned power flow problems," *Int. J. Electr. Power Energy Syst.*, vol. 146, art. 108772, 2023.
- [5] S. Iwamoto and Y. Tamura, "A load flow calculation method for ill-conditioned power systems," *IEEE Trans. Power App. Syst.*, vol. PAS-100, no. 4, pp. 1736–1743, 1981.
- [6] F. Milano, "Continuous Newton's method for power flow analysis," *IEEE Trans. Power Syst.*, vol. 24, no. 1, pp. 50–57, 2009.
- [7] W. F. Tinney and C. E. Hart, "Power flow solution by Newton's method," *IEEE Trans. Power App. Syst.*, vol. PAS-86, no. 11, pp. 1449–1460, 1967.
- [8] B. Stott and O. Alsac, "Fast decoupled load flow," *IEEE Trans. Power App. Syst.*, vol. PAS-93, no. 3, pp. 859–869, 1974.
- [9] R. A. M. van Amerongen, "A general-purpose version of the fast decoupled load flow," *IEEE Trans. Power Syst.*, vol. 4, no. 2, pp. 760–770, 1989.
- [10] R. D. Zimmerman, C. E. Murillo-Sánchez and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, 2011.
- [11] M. Tostado-Véliz, S. Kamel and F. Jurado, *Matpower ill-conditioned systems* (version 1), data set, Zenodo, record 3514739, 2019.
- [12] M. Tostado-Véliz, S. Kamel and F. Jurado, *Matpower limit cases* (version 1), data set, Zenodo, record 3491654, 2019.
- [13] S. Fliscounakis, P. Panciatici, F. Capitanescu and L. Wehenkel, "Contingency ranking with respect to overloads in very large power systems taking into account uncertainty, preventive, and corrective actions," *IEEE Trans. Power Syst.*, vol. 28, no. 4, pp. 4909–4917, 2013.
- [14] L. T. Watson, "Globally convergent homotopy methods: A tutorial," *Appl. Math. Comput.*, vol. 31, pp. 369–396, 1989.
- [15] E. L. Allgower and K. Georg, *Introduction to Numerical Continuation Methods*, SIAM Classics in Applied Mathematics, vol. 45, 2003.
- [16] E. Hairer and G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2nd ed., Springer, 1996.
- [17] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quart. Appl. Math.*, vol. 2, no. 2, pp. 164–168, 1944.
- [18] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *J. SIAM*, vol. 11, no. 2, pp. 431–441, 1963.
- [19] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [20] X. S. Li, "An overview of SuperLU: Algorithms, implementation, and user interface," *ACM Trans. Math. Softw.*, vol. 31, no. 3, pp. 302–325, 2005.

A Reproducing This Report

All results come from the repository. Test systems are downloaded from Zenodo on first use.

Commands

```
pip install -r requirements.txt
python -m pytest tests # 24 tests
python run_all.py # every experiment (1-15)
python run_all.py --only 3 4 # selected experiments
python -m experiments.exp11_improvements --configs PC "OM1+1PC" richardson
python -m improvements --help # any modification on any of the 34 systems
python -m extensions spectrum case18482 # any proposal study on any system
python report/scripts/figures_from_results.py # report figures from results/tables
python report/scripts/figures_from_code.py # report figures that call the solvers
```

Experiments 1 to 10 (the paper) take about an hour on a laptop; experiment 11 (17 configurations $\times$ 170 runs) takes several hours. With `-configs`, experiment 11 runs only the named configurations, always together with the paper’s method, and writes its tables as `improvements_selected_*`.

Table 11. Map from each result of the paper, and of our own studies, to the script that produces it and its output in `results/`.

Result	Script	Output
Figures 1 and 2	exp01_tutorial.py	fig1*.png, fig2*.png
Section 4.2.1	exp02_first_step.py	sec421_first_step.md
Tables 2, 3, 4	exp03_tables2to4.py	table2_FE.md, table3_RK2.md, table4_BE.md
Table 5	exp04_table5.py	table5.md
Section 4.4	exp05_sec44_pathways.py	sec44_pathways.md
Figure 3, Table 6	exp06_fig3_table6.py	fig3*.png, table6.md
Figure 4	exp07_fig4.py	fig4_case109272_states.png
Figure 5	exp08_fig5.py	fig5_timestep_sensitivity.png
Table 7	exp09_table7.py	table7.md, table7_native_base.md
Table 8	exp10_table8.py	table8.md
Our modifications	exp11_improvements.py	improvements_*.md
Proposal: spectrum	exp12_spectrum.py	spectrum.md
Proposal: $(K, \Delta t_0)$ maps	exp13_feasibility.py	feasibility.md, feasibility_maps.png
Proposal: tuning δ	exp14_tuning.py	tuning_per_case.md, tuning_rules.md
Proposal: own LU	exp15_scratch_lu.py	scratch_lu.md, gauss_vs_lu.md

B Full Tables

Table 12. Table 2 re-run: FE after a BE first step, path $\{0, 0.005, 0.01, 0.51, 1\}$, $K = 10^{-4}$ (our value above the paper's).

Case		$t=0$	0.005	0.01	0.51	1	NR 1	NR 2	NR 3	Result
case18482	ours	533	65	3.7	1943	1.3×10^5	2.6×10^6	1.4×10^7	8.1×10^9	no
	paper	532	65	9.7	2000	2000	–	–	–	
case27318	ours	201	25	1.2	27	1.6×10^7	3.3×10^7	7.5×10^7	1.7×10^8	no
	paper	200	25	1.2	27	2×10^7	–	–	–	
case36964	ours	533	66	3.5	1837	2.7×10^5	7.7×10^9	10^{12}	1.3×10^{13}	no
	paper	532	65	3.5	2000	3×10^5	–	–	–	
case54636	ours	201	25	1.2	27	1.1×10^9	9.6×10^9	7.1×10^9	6.4×10^{11}	no
	paper	200	25	1.2	27	10^9	–	–	–	
case109272	ours	201	25	1.2	27	1.1×10^9	9.6×10^9	7.1×10^9	6.4×10^{11}	no
	paper	200	25	1.2	27	10^9	–	–	–	
case69limit	ours	0.012	2.4×10^{-4}	1.2×10^{-5}	5.4×10^{-4}	2.4×10^{-5}	1.7×10^{-8}	4.3×10^{-12}	–	conv. (2)
	paper	0.012	2×10^{-4}	10^{-5}	5×10^{-4}	2×10^{-5}	2×10^{-8}	3×10^{-12}	–	
case141limit	ours	6.4×10^{-3}	1.6×10^{-4}	8.6×10^{-6}	4.6×10^{-4}	2.3×10^{-5}	3.9×10^{-8}	1.7×10^{-11}	–	conv. (2)
	paper	6×10^{-3}	2×10^{-4}	9×10^{-6}	5×10^{-4}	2×10^{-5}	4×10^{-8}	10^{-11}	–	
case500limit	ours	13	1.2	0.022	0.92	1	8.8×10^{-3}	1.9×10^{-6}	2.3×10^{-13}	conv. (3)
	paper	13	1.2	0.02	0.9	1	8×10^{-3}	2×10^{-6}	4×10^{-13}	
case2000limit	ours	55	3.9	0.78	210	6802	3.2×10^7	5.4×10^7	1.2×10^8	no
	paper	54	3.9	0.8	210	7000	–	–	–	

Table 13. Table 3 re-run: RK2 after a BE first step, same path and K .

Case		$t=0$	0.005	0.01	0.51	1	NR 1	NR 2	NR 3	Result
case18482	ours	533	65	33	2.5×10^7	6.8×10^8	1.3×10^{15}	3×10^{15}	6.8×10^{15}	no
	paper	532	65	33	3×10^7	10^{11}	–	–	–	
case27318	ours	201	25	13	5.8×10^5	1.6×10^{11}	4.2×10^{10}	9.4×10^{10}	2.1×10^{11}	no
	paper	200	25	13	6×10^5	10^{10}	–	–	–	
case36964	ours	533	66	33	2.3×10^9	4.3×10^{12}	3×10^{12}	3.2×10^{13}	7.3×10^{13}	no
	paper	532	65	33	3×10^9	2×10^{14}	–	–	–	
case54636	ours	201	25	13	6.9×10^5	1.6×10^{11}	1.3×10^{11}	1.1×10^{11}	2.4×10^{11}	no
	paper	200	25	13	7×10^5	10^{10}	–	–	–	
case109272	ours	201	25	13	1.1×10^7	1.6×10^{11}	1.3×10^{11}	1.1×10^{11}	2.4×10^{11}	no
	paper	200	25	13	10^7	5×10^{12}	–	–	–	
case69limit	ours	0.012	2.4×10^{-4}	1.2×10^{-4}	9.3×10^{-5}	4.7×10^{-5}	1.2×10^{-9}	–	–	conv. (1)
	paper	0.012	2×10^{-4}	10^{-4}	9×10^{-5}	5×10^{-5}	10^{-9}	–	–	
case141limit	ours	6.4×10^{-3}	1.6×10^{-4}	8.2×10^{-5}	4.3×10^{-4}	2.2×10^{-4}	7×10^{-9}	–	–	conv. (1)
	paper	6×10^{-3}	2×10^{-4}	8×10^{-5}	4×10^{-4}	2×10^{-4}	7×10^{-9}	–	–	
case500limit	ours	13	1.2	0.62	1.1×10^4	2.1×10^4	4.8×10^4	1.1×10^5	2.4×10^5	no
	paper	13	1.2	0.6	10^4	5000	–	–	–	
case2000limit	ours	55	3.9	1.9	1.2×10^4	4.5×10^4	10^5	1.1×10^6	4×10^6	no
	paper	54	3.9	1.9	10^4	4×10^4	–	–	–	

Table 14. Table 6 re-run: median CPU time of seven runs as a percentage of NR-MAT, with the NR iterations after the homotopy in brackets (ours / paper).

Method	t_3	case109272	case36964	case2000limit
NR-MAT	–	1.784 s (100%)	0.797 s (100%)	0.024 s (100%)
FE	0.20	143% [3] / 146%	185% [7] / 178%	272% [4] / 68%
FE	0.25	143% [3] / 145%	fail / fail	272% [4] / 75%
FE	0.30	164% [4] / 170%	201% [8] / 179%	239% [3] / 72%
RK2	0.20	225% [4] / 230%	fail / 232%	341% [3] / 82%
RK2	0.25	224% [4] / 226%	222% [6] / 190%	340% [3] / 84%
RK2	0.30	224% [4] / 237%	221% [6] / 195%	342% [3] / 73%
BE	0.20	163% [4] / 171%	168% [6] / 144%	238% [3] / 61%
BE	0.25	164% [4] / 171%	169% [6] / 144%	237% [3] / 81%
BE	0.30	163% [4] / 169%	170% [6] / 148%	238% [3] / 94%